Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____

КАФЕДРА _____

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ

НА ТЕМУ:

_ Рекомендательная система по подбору оправы для очков _____

_____

_____

_____

_____

_____

Студент ___ИУ5-24М____ _____ ____**Д.О. Ишков**____
(Группа) (Подпись, дата) (И.О.Фамилия)

Руководитель курсового проекта _____ _**М. В. Виноградова**__
(Подпись, дата) (И.О.Фамилия)

Консультант _____ _____
(Подпись, дата) (И.О.Фамилия)

2021 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой _____
(Индекс)
_____ _____
(И.О.Фамилия)
« _____ » _____ 20 ____ г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине ____Технологии разработки программного обеспечения_____

Студент группы ___ИУ5-24М_____

_____Ишков Денис Олегович_____
(Фамилия, имя, отчество)

Тема курсового проекта _____ Рекомендательная система по подбору оправы для очков _____
_____
_____

Направленность КП (учебный, исследовательский, практический, производственный, др.)
_____
Источник тематики (кафедра, предприятие, НИР) _____

График выполнения проекта: 25% к 4 нед., 50% к 8 нед., 75% к 12 нед., 100% к 15 нед.

Задание Выполнить разработку системы в соответствии с описанием ее функциональности на основе моделей унифицированного процесса (RUP). Написать программу, реализующую фрагмент системы и реализовать в ней паттерны бизнес-логики, работы с БД и gof («active record» и «table module»)

Оформление курсового проекта:

Расчетно-пояснительная записка на _67_ листах формата А4.
Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.)
_____
_____
_____
_____

Дата выдачи задания « 7 » __ февраля___ 2021 г.

Руководитель курсового проекта _____ __М. В. Виноградова__
(Подпись, дата) (И.О.Фамилия)

Студент _____ ____Д. О. Ишков_____
(Подпись, дата) (И.О.Фамилия)

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

Оглавление

1. Этап анализа и планирования требований 5
1.1. Перечень функциональных и нефункциональных требований 5
1.2. Модель предметной области 8
1.3. Выявленные актёры 8
1.4. Выявленные прецеденты, их приоритеты и описание 9
1.5. Диаграмма прецедентов 10
1.6. Описание возможной архитектуры 11
1.7. Диаграммы уровней подсистем и развертывания 12
1.8. Перечень критических рисков 13
1.9. Перечень экранных форм и их сложность 14
1.10. Экспертные оценки скорости разработки и масштабных факторов 17
1.11. Затраты, длительность и стоимость разработки 19
2. Этап проектирования 20
2.1. Уточненная диаграмма прецедентов 20
2.2. Расширенное описание основных прецедентов 21
2.3. Прототип пользовательского интерфейса 24
2.4. Диаграммы классов анализа: граничных, управляющих, сущностей 26
2.5. Диаграммы взаимодействия для основных прецедентов 28
2.6. Пакеты анализа и сервисные пакеты в форме обобщенной диаграммы классов 31
2.7. Трассировка пакетов в подсистемы 32
2.8. Трассировка классов анализа в классы проектирования 33
2.9. Подсистемы, их уровни и зависимости 34
2.10. Диаграмма развертывания, узлы и линии связи, с указанием размещенных подсистем 35
2.11. Трассировка подсистем в компоненты 36
2.12. Трассировка классов проектирования в исходные файлы 36
2.13. Зависимость компонентов от исходных файлов 38
2.14. Диаграммы компонентов и развертывания для реализации архитектуры 39
2.15. Переработанный список рисков 40
2.16. Перечень итераций и их состав 41

3. Этап построения 44
3.1. Диаграммы классов с учетом реализованных паттернов 44
3.2. Диаграмма последовательностей в терминах подсистем 48
3.3. Диаграмма последовательностей в терминах классов проектирования ... 49
3.4. Исходный код программы, реализующей архитектурно-значимые прецеденты 50
3.5. Описание тестовых вариантов для тестирования модулей 55
3.6. Перечень и последовательность проведения тестов 60
4. Этап внедрения 66
4.1. Перечень программ и рекомендации по установке 66
4.2. Перечень документации для пользователей и заказчиков 66
4.3. Рекомендации по внедрению 66
Список использованных источников 67

1. Этап анализа и планирования требований

Таблица 1. Постановка задачи (задание по варианту)

Тема	Паттерн бизнес-логики	Паттерн работы с БД	Паттерн GOF
Рекомендательная система по подбору оправы для очков	table module	active record	компоновщик

1.1. Перечень функциональных и нефункциональных требований

Функциональные:

Рекомендательная система по подбору оправы для очков должна выполнять следующие функции:

- Система должна позволять пользователю настраивать параметры для рекомендации;
- Система должна позволять пользователю загружать изображение с лицом;
- Система должна позволять пользователю выбирать метод рекомендаций;
- Система должна позволять изменять язык, на котором будут написаны рекомендации;
- Система должна позволять связываться пользователю/консультанту с администратором по конкретному вопросу;
- Система должна позволять пользователю просматривать результат рекомендации системы;
- Система должна позволять пользователю просматривать объяснения рекомендаций системы;
- Система должна позволять пользователю просматривать перечень черт внешности;
- Система должна позволять пользователю просматривать подобранные модели оправ;

- Система должна позволять консультанту просматривать правила подбора оправ;
- Система должна позволять администратору редактировать правила подбора оправ.

Нефункциональные:

Ограничения среды и реализации:

- Система должна разрабатываться на ЯП Python версии 3.7.0.

Требования по производительности:

- Среднее время формирования рекомендаций для пользователя должно быть минимальным (не более 5 секунд на запрос);
- Для работы системы необходимо не более 2ГБ оперативной памяти;
- Для работы системы требуется пропускная способность интернет соединения на не менее 50кбит/с.

Требования по зависимости от платформы:

- Система должна работать на ОС Windows и Linux.

Требования по надежности:

- Система должна кешировать запросы для уменьшения количества обращений к внешним сервисам.

Требования к пользовательскому интерфейсу:

- Дизайн системы должен быть лаконичен и понятен любому пользователю.

Требования к форматам и протоколам передачи данных:

- Запросы к внешним сервисам должны осуществляться при помощи сетевого протокола HTTP версии 1.1.

Требования к наличию внешних компонентов или подсистем:

- Для работы системы используются внешние сервисы распознавания лиц: FacePlusPlus и BetaFace.

1.2. Модель предметной области

Была составлена модель предметной области, она же диаграмма классов, которая представлена на рисунке 1.

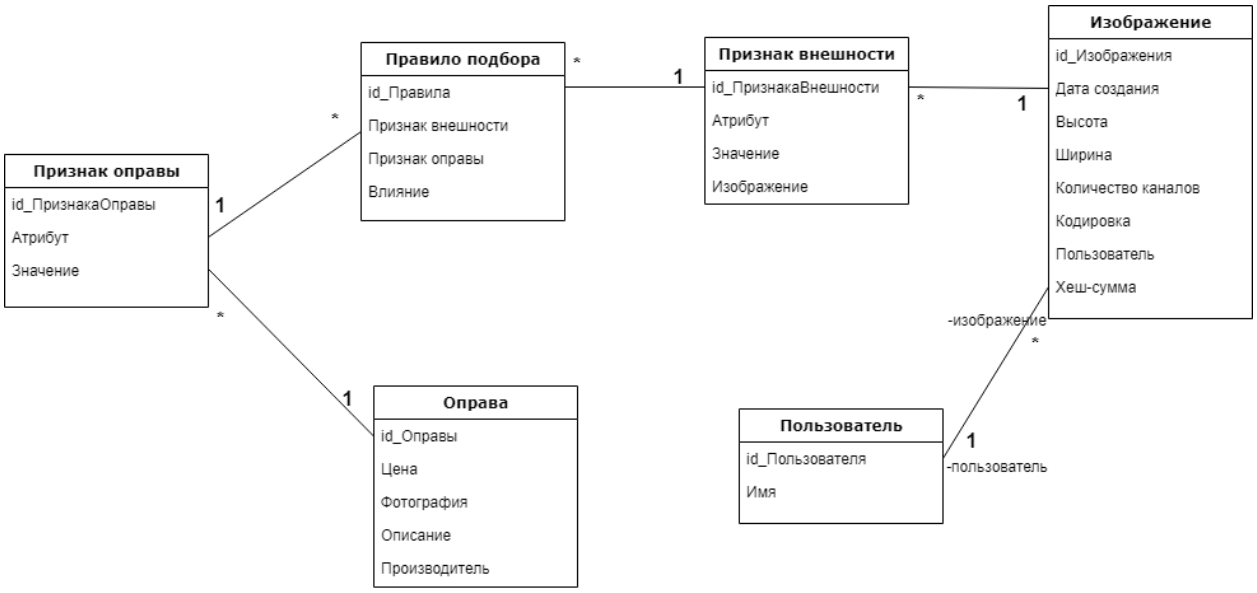


Рисунок 1. Модель предметной области

1.3. Выявленные актёры

Были выявлены следующие актеры:

1. Консультант. Он может консультировать клиента, заносить данные о Клиента, в частности его фотографии, подбирать оправы по базе, просматривать правила, по которым осуществляется подбор;

2. Администратор. Он использует систему для редактирования правил подбора оправ;

3. Клиент или Пользователь осуществляет взаимодействие с системой через Консультанта.

1.4. Выявленные прецеденты, их приоритеты и описание

Были сформулированы следующие приоритеты:

1. Загрузка изображения с лицом – загружает цифровую фотографию с изображением Пользователя в систему, при этом выделяет из неё признаки внешности. Приоритет – высокий.
2. Выбор метода рекомендации – настройка для выбора способа поиска подходящих оправ. Приоритет – низкий.
3. Выбор языка интерфейса системы – настройка отображения графического интерфейса. Приоритет – низкий.
4. Просмотр подходящих оправ – отображение фотографий моделей оправ, подходящих по внешности к Пользователю. Приоритет – высокий.
5. Просмотр перечня черт внешности – отображение списка распознанных черт внешности Пользователя из фотографии. Приоритет – низкий.
6. Просмотр объяснения рекомендаций – отображение логики подбора в виде текстового описания сработанных правил. Приоритет – средний.
7. Связаться с администратором – отправить заявку для контакта с администратором, чтобы разрешить те или иные вопросы. Приоритет – низкий.
8. Просмотр правил подбора оправ – отображение таблицы со столбцами атрибут Признака внешности, значение атрибута внешности, атрибут Признака оправы, значение атрибута оправы, степень влияния, где степень влияния отображает вклад Признака Внешности в конечный визуальный вид Оправы. Приоритет – средний.
9. Редактирование правил подбора оправ – редактирование, добавление или удаление правил подбора оправ через интерфейс системы. Приоритет – низкий.

1.5. Диаграмма прецедентов

Была составлена диаграмма по выявленным ранее прецедентам, диаграмма представлена на рисунке 2.

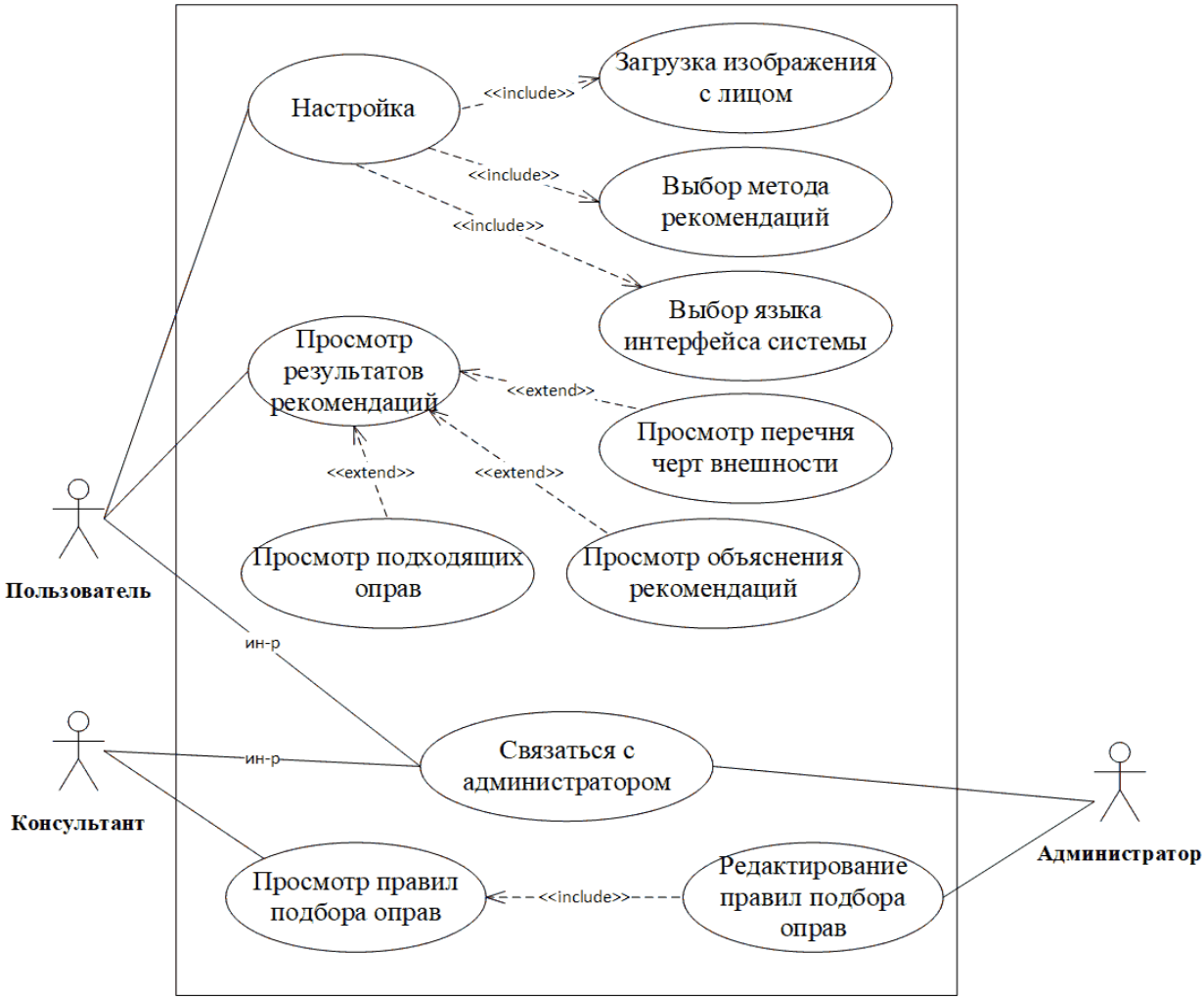


Рисунок 2. Диаграмма прецедентов

1.6. Описание возможной архитектуры

Архитектурно-значимые прецеденты:

1. Загрузка изображения с лицом;
2. Просмотр подходящих оправ.

Возможные архитектурные решения:

1. Организация БД и СУБД;
2. Разработка приложения для PC с графическим интерфейсом.

Решение архитектурных задач в аналогах

1. Организация БД и СУБД

Перед выбором БД и СУБД надо решить будет использоваться реляционная или не реляционная БД.

Реляционные БД хранят структурированные данные, которые обычно представляют объекты реального мира. Скажем, это могут быть сведения о человеке, или о содержимом корзины для товаров в магазине, сгруппированные в таблицах, формат которых задан на этапе проектирования хранилища [2].

Нереляционные БД устроены иначе. Например, документо-ориентированные базы хранят информацию в виде иерархических структур данных. Речь может идти об объектах с произвольным набором атрибутов. То, что в реляционной БД будет разбито на несколько взаимосвязанных таблиц, в нереляционной может храниться в виде целостной сущности.

В качестве конкретного примера СУБД можно привести SQLite. Эта СУБД имеет синтаксис, который прост в освоении, также она обладают большим быстродействием.

2. Разработка приложения для PC с графическим интерфейсом

Так как в качестве нефункциональных требований указано, что система должна запускаться на разных операционных устройствах (Windows и Linux), то для разработки имеет смысл использовать кроссплатформенный язык программирования, чтобы не разрабатывать отдельное приложение под каждую систему. Конечно, можно выбрать Java, однако может возникнуть проблема отсутствия реализаций тех или иных компонентов, поэтому разработка может затянуться. Поэтому предлагается использовать интерпретируемый язык Python, который лишён данной проблемы. В качестве графического интерфейса существует множество решений, например PyQt, PySide, Kivy, есть также и поддержка веб-интерфейсов. Однако разработка интерфейса с помощью веб-интерфейса требует навыков работы HTML, CSS, JavsScript, тем самым повышаются требования к навыкам разработчиков. Чтобы этого избежать, предлагается использовать библиотеку PySimpleGUI.

1.7. Диаграммы уровней подсистем и развертывания

Была сформирована возможная диаграмма уровней подсистем (см. рисунок 3), а также упрощенная диаграмма развёртывания (см. рисунок 4).

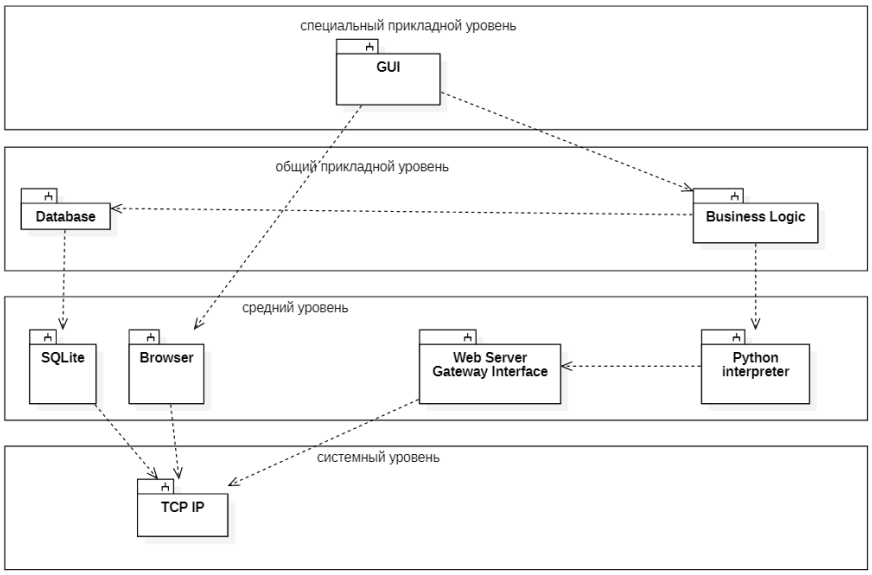


Рисунок 3. Возможная диаграмма уровней подсистем

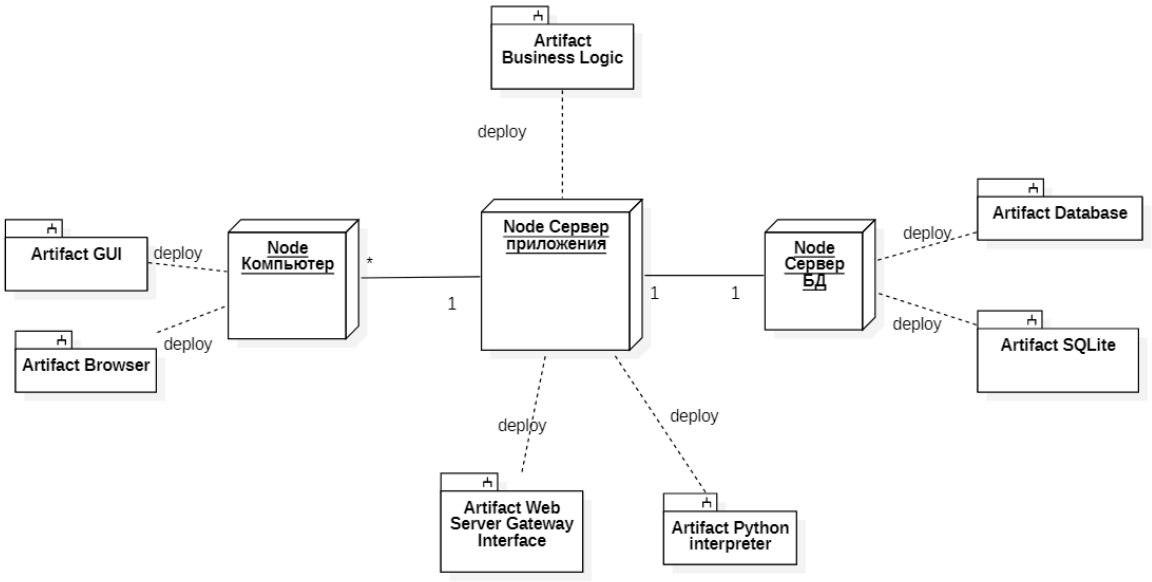


Рисунок 4. Упрощенная диаграмма развертывания

1.8. Перечень критических рисков

В таблице 2 представлены риски и рекомендации по их управлению.

Таблица 2. Риски и рекомендации по их управлению

Риск	План управления
Критическая программная ошибка	Поиск и устранение ошибки
Ошибка потери пользовательских данных	Поиск возможностей по созданию бекапов данных и дальнейшему восстановлению из них
Ошибки преобразования данных приводят к избыточным вычислениям	Приведение типов с плавающей точкой к типам с фиксированной точкой для контроля погрешности вычислений
Низкая скорость отклика системы	Поиск возможностей по оптимизации работы системы
Недостаточно понятный и малоэффективный интерфейс пользователя	Проведение А/Б тестирования для выявления простого и понятного интерфейса
Отставание по срокам	Привлечение дополнительной рабочей силы
Потеря финансирования	Поиск новых инвесторов
Создание продукта, не требующегося на рынке	Прогнозирование необходимости продукта в будущем, в случае отсутствия каких-либо перспектив прекращение разработки

1.9. Перечень экранных форм и их сложность

Были выявлены следующие формы:

1. Форма изменения языка интерфейса системы (представление экрана №1)
   1.1 Функция изменения языка интерфейса системы:
      - Поля для ввода: раскрывающийся список поддерживаемых языков;
      - Кнопки: изменить язык системы, отменить.
2. Форма для обратной связи с администратором (представление экрана №1)
   2.1 Функция заполнения формы пользователем системы с целью связи с администратором по конкретному вопросу:
      - Поля для ввода: ФИО, адрес почты, вопрос;
      - Кнопки: кнопка для отправки вопроса.
3. Форма с предложенными оправами для пользователя системы (представление отчёта №1)
   3.1 Функция просмотра пользователем системы перечня черт внешности:
      - Таблица, столбцы: атрибут внешности, значение атрибута, описание подходящих оправ;
      - Кнопки: назад.
   3.2 Функция просмотра пользователем системы подобранных моделей оправ
      - Таблица, столбцы: изображение оправ, модель, краткое описание, альтернативные модели;
      - Кнопки: назад.

4. Форма получения рекомендаций системы (представление отчёта №1)

   4.1 Функция получения рекомендаций системы:

   - Поля для ввода: фотография с лицом пользователя, способ подбора (раскрывающийся список);
   - Кнопки (ссылки): подобрать (Предложенные оправы).

5. Форма просмотра правил подбора оправ (представление отчета №2)

   5.1 Функция просмотра правил подбора оправ:

   - Таблица, столбцы: атрибут внешности, атрибут оправы, степень влияния;
   - Кнопки: группировка по каждому столбцу, назад.

6. Форма редактирования/удаления правил подбора оправ (доступна администратору) (представление экрана №2)

   6.1 Функция редактирования правил подбора оправ:

   - Таблица, столбцы: атрибут внешности, атрибут оправы, степень влияния;
   - Кнопки: сохранить, отменить, группировка по каждому столбцу.

Таблица 3. Оценка сложности формы и отчёта по методологии COCOMO II (метод объектного указателя) [3].

Количество представлений и секций на экранной форме		**Количество клиентских и серверных таблиц**		
Сложность экрана		< 4, < 2 серверов, < 3 клиентских таблиц	< 8, 2 - 3 сервер., 3 – 5 клиен. таблиц	>= 8, > 3 серввер., > 5 клиен. таблиц
	< 3	Простой	Простой	Средний
	3 - 7	Простой	Средний	Сложный
	> 8	Средний	Сложный	Сложный
Сложность отчета	0 или 1	Простой	Простой	Средний
	2 - 3	Простой	Средний	Сложный
	>= 4	Средний	Сложный	Сложный

Таблица 4. Оценка количества объектных указателей (ОР) [3]

Типы объекта	Кол-во	Вес			Итог
		Простой	Средний	Сложный	
Экран	-	X1	X2	X3	=_
Отчёт	-	X2	X5	X8	=_
3GL компоненты	-			X10	=_
Объектные указатели					=_

Для настройки и загрузки фотографии пользователь может находиться на одном экране №1 с двумя представлениями (формами). Для загрузки используется 3 таблицы БД, отсюда следует, что Сложность экрана будет *Простой* (см. таблицу 3).

Для редактирования признаков есть один экран №2 с одной формой. Для загрузки используется 3 таблицы БД, отсюда следует, что Сложность экрана будет *Простой* (см. таблицу 3).

Для пользователя существует 2 отчёта (№1 и №2), для первого два представления, для второго – одно. Для первого из них происходит взаимодействие с 8 таблицами БД, для второго с 5. Отсюда следует, что первый отчёт имеет *Сложную* сложность, а второй – *Простую*.

Используя таблицу 4, получаем:

- 1-ый экран оценивается 1 ОР;
- 2-ой экран оценивается 1 ОР;
- 1-й отчёт оценивается 8 ОР;
- 2-й отчёт оценивается 2 ОР.

Общая оценка количества ОР равна двенадцати (ОР=12).

1.10. Экспертные оценки скорости разработки и масштабных факторов

Реальная опытность команды разработки была средней, поскольку весь проектируется одним человеком. При этом опыт разработки имеется, поскольку команда хорошо знакома с Языком Программирования, на котором будет вестись разработка. Поэтому производительности разработки номинальная, что, в терминах объектных указателей равно тринадцати (**PROD = 13**).

Количество объектных указателей равно 12 (**OP = 12**).

В проекте не происходит повторного использования ресурсов, поэтому коэффициент REUSE = 0%, поэтому **NOP = OP**.

Всего существует 5 параметров W_i (масштабных факторов), поэтому оценим каждый из них:

1. Предсказуемость PREC

Для моей команды уже ранее решались похожие задачи, поэтому **PREC = 3.**

2. Гибкость разработки FLEX

Были заданные общие черты с некоторыми чёткими ограничениями и послаблениями, поэтому **FLEX =4.**

3. Разрешение архитектуры/риска RESL

Был проведен малый анализ риска, поэтому **RESL = 4.**

4. Связность группы TEAM

Связность высокая, поэтому **TEAM = 2**.

5. Зрелость процесса PMAT

Зрелость среднего уровня, вычисленная по вопроснику CMM, поэтому **PMAT = 3**.

PREC является базовым параметром модели композиции приложения, поэтому его значение напрямую влияет на параметры FLEX, RESL и PMAT, поскольку опытность команды в других проектах, другой организации и другом планировании в полной степени отражается на при способности организовывать работу над текущим проектом. Поэтому значение PREC=3 задало значение 3 для всех трех перечисленных выше параметров.

Однако параметр слаженности команды может идти в абсолютном отрыве от остальных параметров, и любое его значение может абсолютно не повлиять на них.

Стоит заметить, что параметр FLEX имеет внешний характер относительно команды, поэтому его значение может также сильно влиять на работу команды, как и параметр PREC. Иными словами, параметр PREC тоже может менять значение в зависимости от FLEX.

Итого, можно сделать вывод, что каждый параметр модели по-своему влияет на общие характеристики модели, каждый из них необходим и важен для анализа будущего процесса работы над проектом, контроля и оптимизации этого процесса.

1.11. Затраты, длительность и стоимость разработки

Затраты [1]:

$$\text{ЗАТРАТЫ} = \frac{NOP}{PROD} = \frac{12}{13} = 0{,}92 \quad (1)$$

Длительность [1]:

$$(TDEV) = \left[3 * (\text{ЗАТРАТЫ})^{(0{,}33+0{,}2*(B-1{,}01))}\right] * \frac{SCEDPercentage}{100} [\text{мес}] \quad (2)$$

$SCEDPercentage$ меняется от 75 до 160% (растёт при усложнении графика работ). Так как команда не имеет большого опыта работы и планирования, то график работ неизбежно будет усложняться от этапа к этапу, и вполне возможно, что команда будет проходить одни и те же этапы более одного раза. Поэтому $\boldsymbol{SCEDPercentage = 140\%}$.

Значение показателя степени B изменяется в диапазоне [1.01, 1.26], зависит от пяти масштабных факторов Wi и вычисляется по формуле.

$$B = 1{,}01 + 0{,}01 * \sum_{i=1}^{5} W_i \quad (3)$$

$$B = 1{,}01 + 0{,}01 * \sum_{i=1}^{5} W_i = 1{,}01 + 0{,}01 * (3 + 4 + 4 + 2 + 3) = 1{,}17 \quad (4)$$

$$(TDEV) = 3 * 0{,}92^{0{,}362} * \frac{140}{100} = 4{,}1 \text{ [мес]} \quad (5)$$

Стоимость [1]:

$$\text{СТОИМОСТЬ} = \text{ЗАТРАТЫ} * \text{РАБ_КОЭФ} = 0{,}92 * 110 = 101 \text{ [тыс. руб.]} \quad (6)$$

где РАБ_КОЭФ = 110 [тыс. руб.], средняя зарплата одного сотрудника в месяц.

2. Этап проектирования

2.1. Уточненная диаграмма прецедентов

Была уточнена диаграмма прецедентов, она представлена на рисунке 5. Были выявлены зависимости включения и расширения между прецедентами. Основными прецедентами являются «Загрузка изображения с лицом», «Просмотр подходящих оправ» и «Редактирование правил подбора оправ». Напомним, что взаимодействия пользователя с системой происходит через Консультанта.

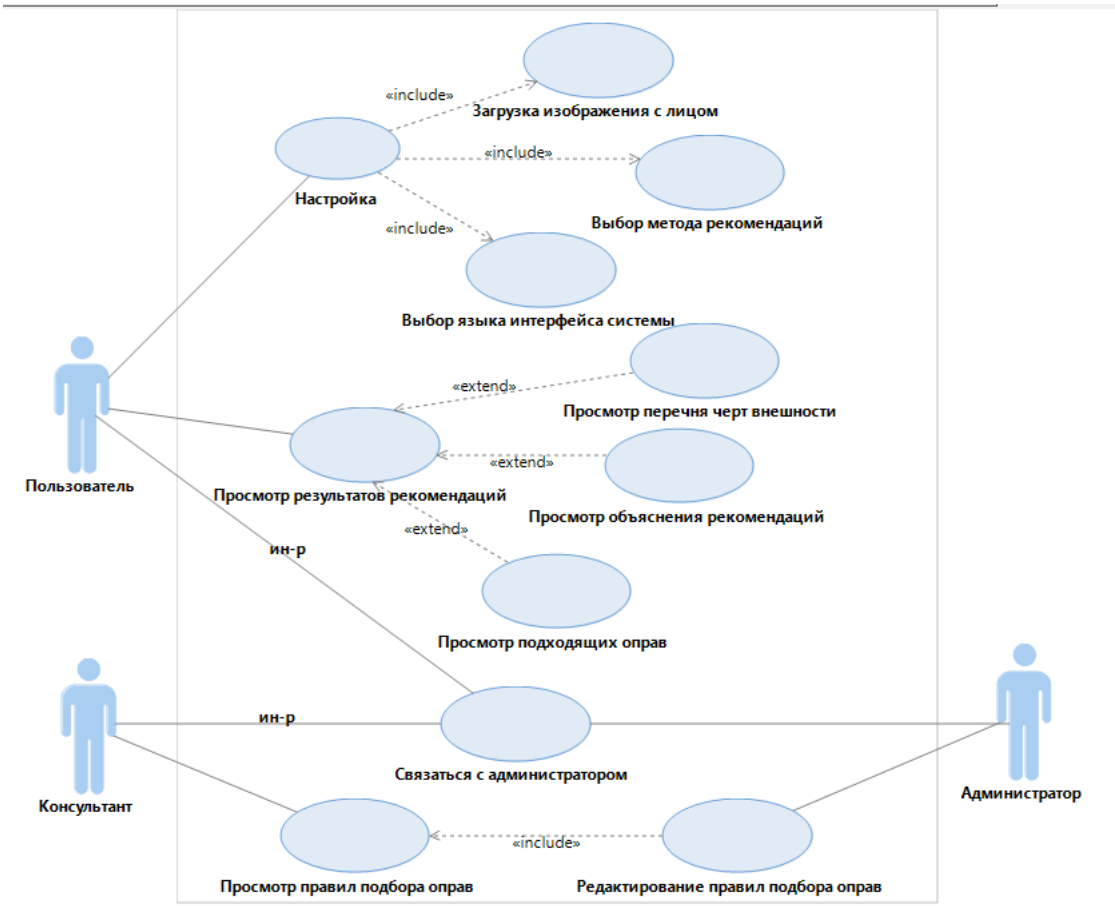


Рисунок 5. Уточненная диаграмма прецедентов

2.2. Расширенное описание основных прецедентов

Было разработано расширенное описание основных прецедентов в виде спецификации. В таблице 5 представлен пример спецификации прецедента.

Таблица 5. Спецификация основного прецедента «Загрузка изображения с лицом»

Прецедент: «Загрузка изображения с лицом»
Краткое описание: Предназначен для загрузки изображения с лицом и признаков в систему
Главные актёры: Пользователь
Второстепенные актёры: Нет
Предусловие: 1. Пользователь должен загрузить систему 2. Выполнен прецедент «Настройка»
Основной поток: 1. Пользователь указывает путь к изображению с лицом 2. Система проверяет формат файла изображения (если плохой - E1) 3. Система считывает и декодирует в массив изображения по заданному пути 4. Система получает ключ массива с помощью алгоритма хеширования md5 5. Система не находит записей в своей хэш-таблице по полученному ключу (иначе A1) 6. Система кодирует массив в BASE64 строку 7. Система отправляет BASE64 строку в сервис распознавания с помощью HTTP POST-запроса 8. По ответу сервиса система ищет местоположение лица на изображении (не найдено - E2) 9. Система выравнивает и обрезает лицо на изображении 10. Система кодирует полученный массив в BASE64 строку 11. Система отправляет BASE64 строку во второй сервис распознавания с помощью HTTP POST-запроса 12. По ответу сервиса система формирует признаки черт внешности 13. Система увеличивает внутренний счетчик числа обращений к сервисам 14. Система сохраняет признаки по ключу массива из ш.4 15. Система отображает пользователю сообщение об успешно загруженной фотографии

Постусловие: Пользователь может загрузить новую фотографию или выйти из системы.
Альтернативные потоки: Альтернативный поток E1**:** 1. Пользователь указывает путь к изображению с лицом 2. Система проверяет формат файла изображения 3. Система выдаёт ошибку пользователю о некорректном формате изображения Альтернативный поток A1**:** 1. Пользователь указывает путь к изображению с лицом 2. Система проверяет формат файла изображения 3. Система считывает и декодирует в массив изображения по заданному пути 4. Система получает ключ массива с помощью алгоритма хеширования md5 5. Система находит запись в своей хэш-таблице по полученному ключу 6. Система отображает пользователю сообщение об успешно загруженной фотографии Альтернативный поток E2**:** 1. Пользователь указывает путь к изображению с лицом … 8. По ответу сервиса система ищет местоположение лица на изображении 9. Система выдаёт ошибку пользователю об отсутствии лица на фотографии

Спецификацию основного прецедента «Загрузка изображения с лицом» можно увидеть в таблице 5. Для этого прецедента были выявлены один основной поток и три альтернативных. Описание прецедента «Просмотр подходящих оправ» приведено в таблице 6. Данный прецедент имеет один основной поток и один альтернативный.

Таблица 6. Спецификация основного прецедента «Просмотр подходящих оправ»

Прецедент: «Просмотр подходящих оправ»
Краткое описание: Предназначен для отображения пользователю 6 лучших оправ из ассортимента магазина с учетом его внешности
Главные актёры: Пользователь
Второстепенные актёры: Нет
Предусловие: 1. Пользователь должен загрузить систему 2. Выполнен прецедент «Загрузка изображения с лицом»
Основной поток: 1. Прецедент начинается после выбора пользователем 2. Система считывает закрепленный за пользователем ключ 3. Система не находит признаки оправ по заданному ключу в хеш-таблице (иначе А1) 4. Система находит признаки внешности по заданному ключу в хеш-таблице 5. Система формирует признаки оправ путём последовательного применения правил к признакам внешности из БД с правилами подбора 6. Система векторизует признаки оправ 7. Система сохраняет вектор признаков оправ в хеш-таблицу по ключу 8. Система считает косинусное расстояние между вектором оправы и векторами моделей очков из БД 9. Система находит 6 самых близких по косинусному расстоянию оправ 10. Система совершает запрос к БД для получения сведений о моделях 11. Система отображает информацию о подходящих моделях пользователю
Постусловие: Пользователь может выйти из системы.
Альтернативные потоки: Альтернативный поток А1**:** 1. Прецедент начинается после выбора пользователем 2. Система считывает закрепленный за пользователем ключ 3. Система находит признаки оправ по заданному ключу в хеш-таблице 4. Система считает косинусное расстояние между вектором оправы и векторами моделей очков из БД 5. Система находит 6 самых близких по косинусному расстоянию оправ 6. Система совершает запрос к БД для получения сведений о моделях 7. Система отображает информацию о подходящих моделях пользователю в виде таблицы

2.3. Прототип пользовательского интерфейса

Прототип пользовательского интерфейса приведён на рисунках 6, 7, 8 и 9. На рисунке 6 показана первоначальная форма настроек рекомендаций с возможностью загрузки изображения. На рисунке 7 продемонстрирована форма с выделенными лицевыми признаками из соответствующей фотографии. На рисунке 8 показаны подобранные подходящие по лицевым признакам оправы. Рисунок 9 показывает описание рекомендаций системы.

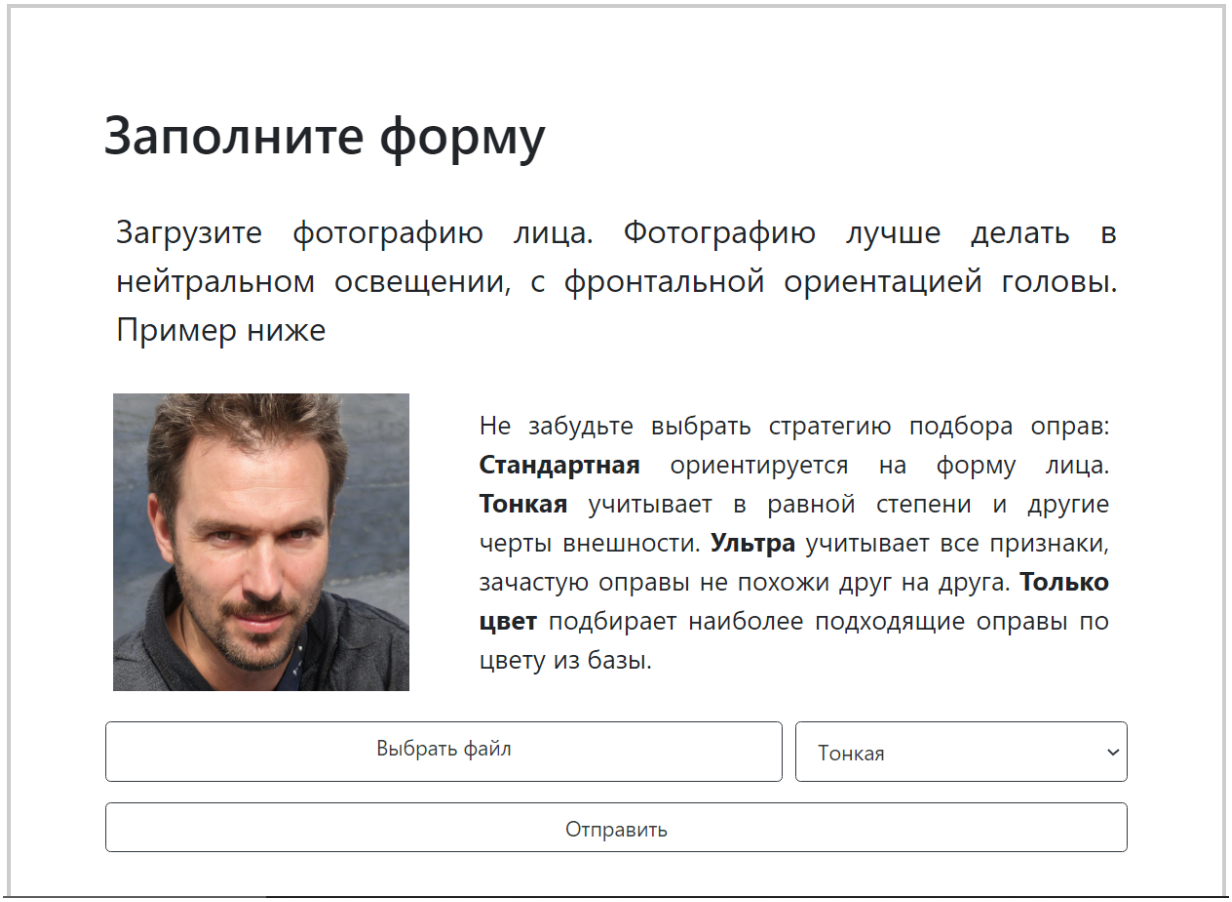


Рисунок 6. Страница настроек



Описание | Подобранные оправы

Форма лица | Пропорции лица | Форма челюсти | Брови | Размер Носа | Глаза | Лоб | Губы | Волосы | Тон кожи | Пол

Женский

Согласно социологическому исследованию, девушки отдают предпочтение ободковым толстым пластиковым оправам.

Рисунок 7. Форма отчета выделенных признаков внешности

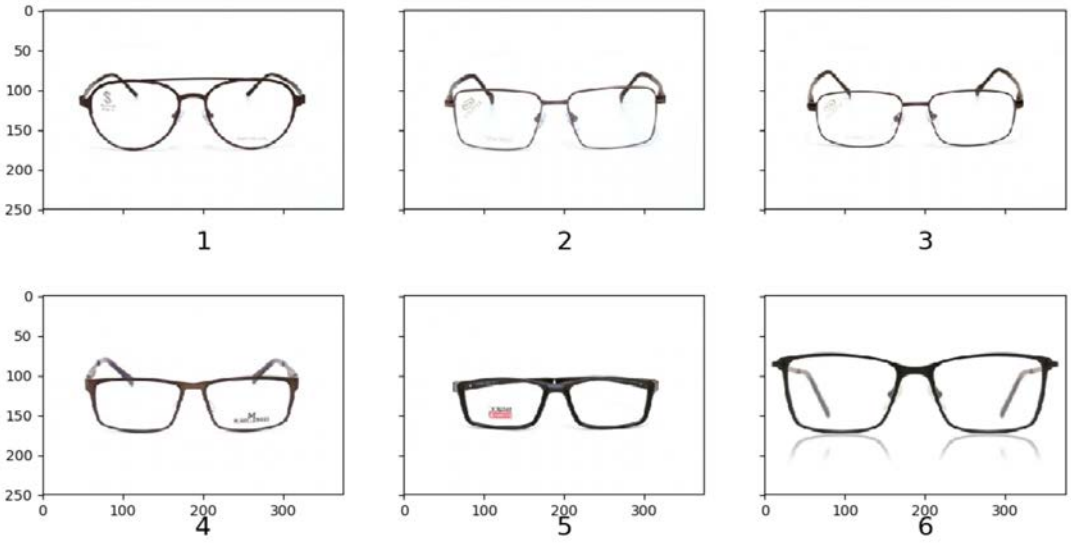


Рисунок 8. Форма отчёта просмотра подходящих оправ

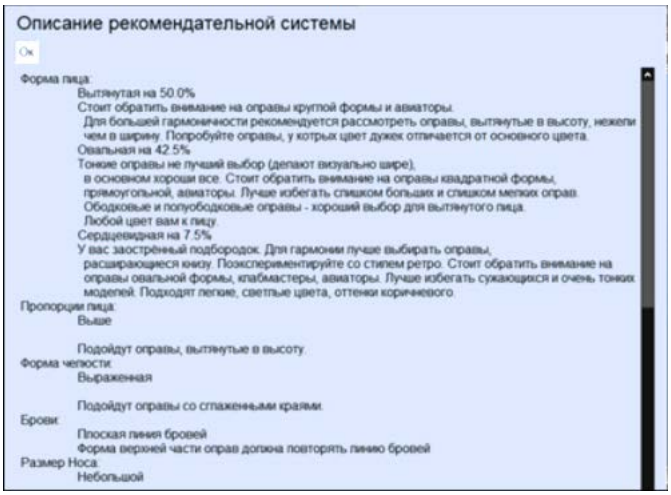


Рисунок 9. Форма отчета объяснения рекомендаций системы

2.4. Диаграммы классов анализа: граничных, управляющих, сущностей

Классы сущностей представлены на рисунке 10. Всего было выявлено 7 первоначальных классов сущностей: Пользователь, Изображение, Признак внешности, Правило подбора, Объяснение правила, Признак оправы и Оправа. Под Пользователем понимается клиент магазина оптики. Изображение – артефакт, который проецирует визуальную внешность Пользователя на фотографию и, таким образом, обеспечивает его описание. В контексте рекомендаций оправ изображение представляет собой цифровой файл, содержащий лицо Пользователя во фронтальном положении.

Признак внешности – артефакт, который задаёт определённую черту или деталь лица из Изображения, таким образом, обеспечивает его описание. В контексте рекомендаций оправ Признак внешности может определять пропорции частей лица, его цветовой тон, а также прическу.

Оправа – артефакт, рамка, оболочка, в которую что-то вделывается. В контексте рекомендаций оправ оправа представляет физический объект для фиксации линз заданной формы, цвета и материала, созданный определённым производителем.

Признак оправы – артефакт, который проецирует визуальный вид Оправы и, таким образом, обеспечивает её описание. В контексте рекомендаций оправ Признак оправы может определять пропорции частей оправы, её толщину, раскраску, а также материал.

Правило подбора – артефакт, который преобразует пару (атрибут Признака внешности, значение атрибута) в тройку (атрибут Признака оправы, значение атрибута, степень влияния), где степень влияния отображает вклад Признака Внешности в конечный визуальный вид Оправы.

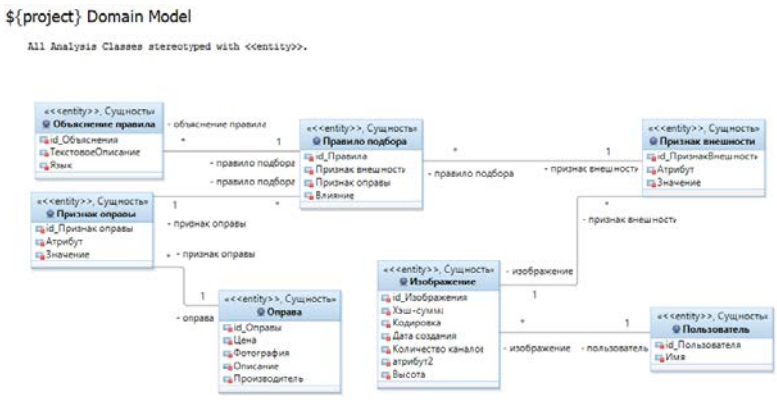


Рисунок 10. Диаграмма классов сущностей

Управляющие классы представлены на рисунке 11. Граничные классы приведены на рисунке 12.



Рисунок 11. Диаграмма управляющих классов

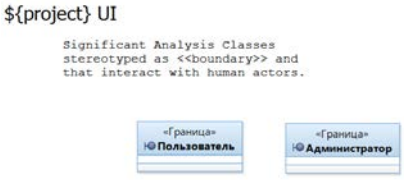


Рисунок 12. Диаграмма граничных классов

2.5. Диаграммы взаимодействия для основных прецедентов

Были сформированы диаграммы деятельности для прецедентов «Загрузка изображения с лицом» (см. рис. 13) и «Просмотр подходящих оправ» (см. рис. 14).

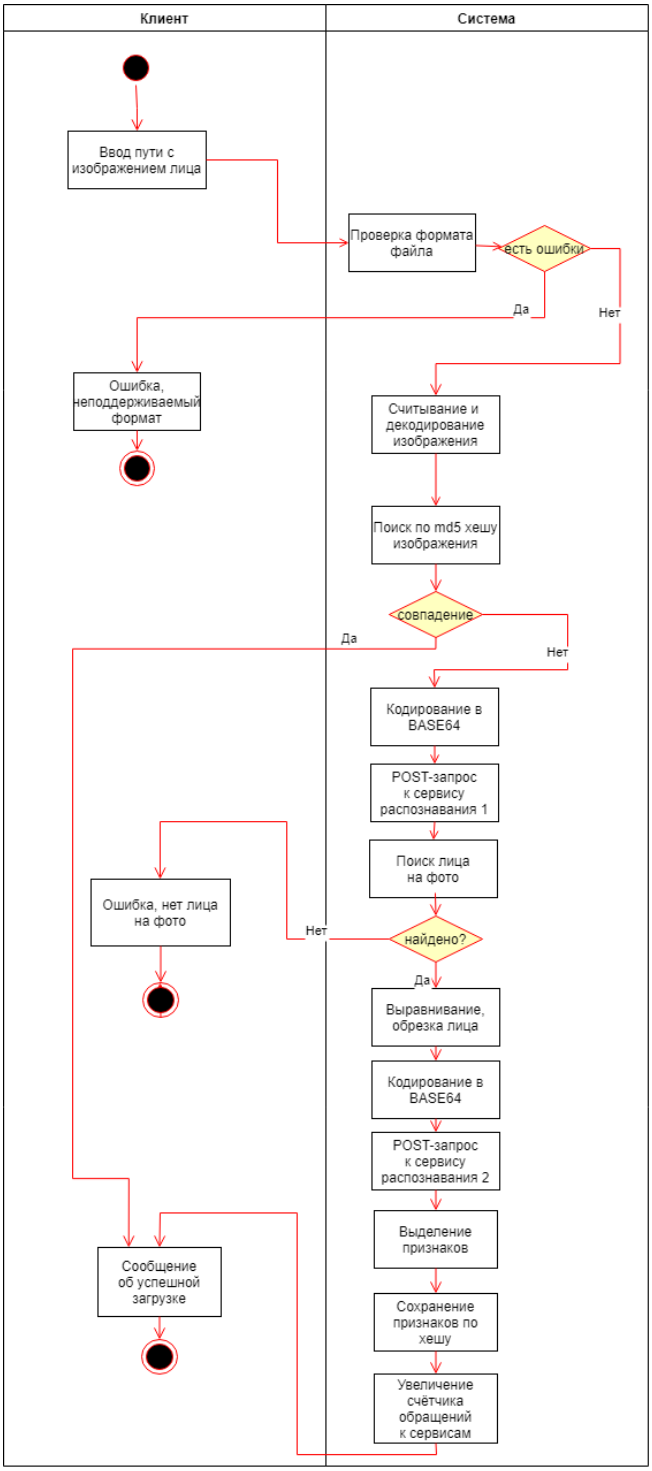


Рисунок 13. Диаграмма деятельности для прецедента «Загрузка изображения с лицом»

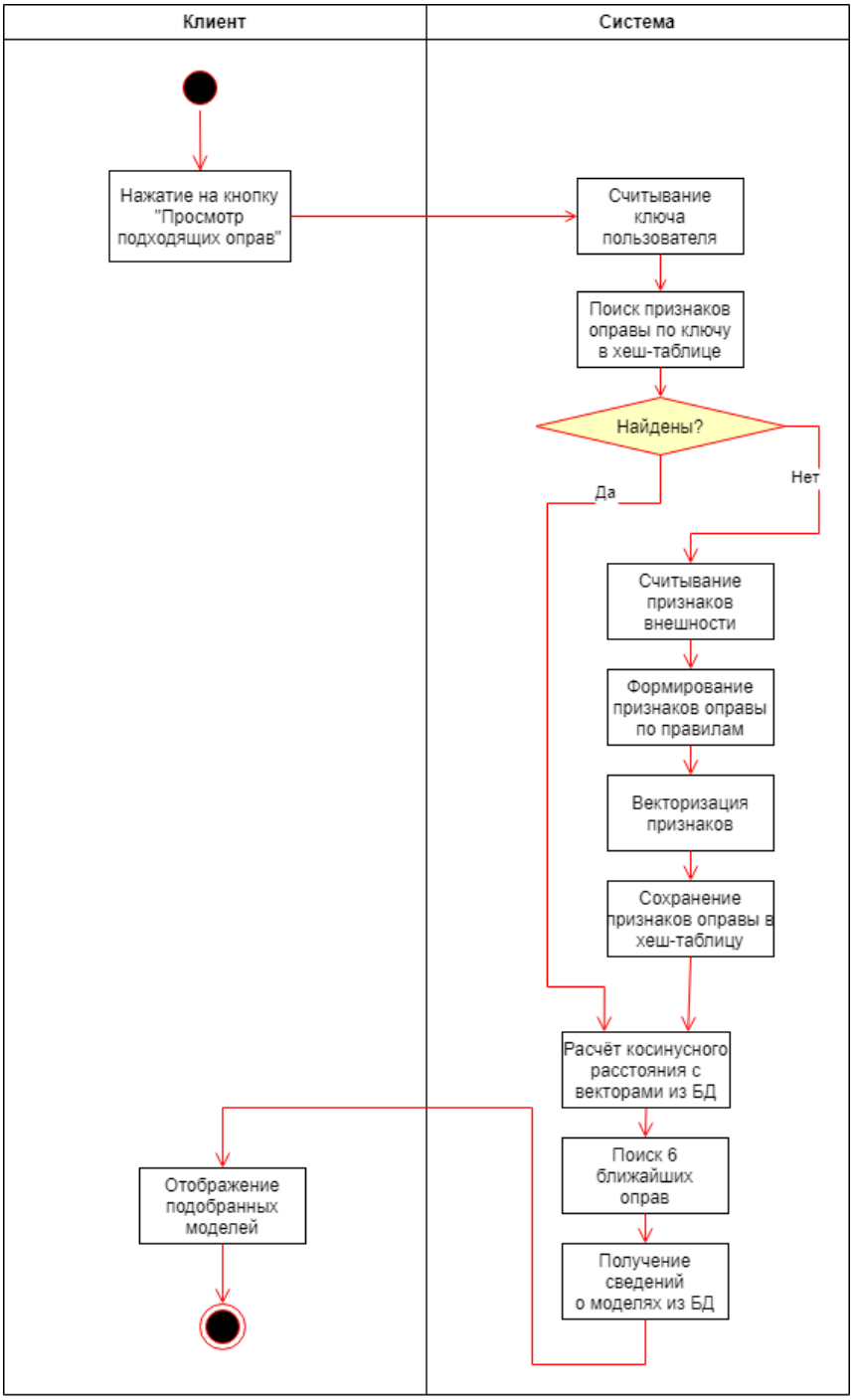


Рисунок 14. Диаграмма деятельности для прецедента «Просмотр подходящих оправ»

Для основных прецедентов «Загрузка изображения с лицом», «Просмотр подходящих оправ» и «Редактирование правил подбора оправ» были также созданы диаграммы взаимодействия, соответственно рисунки 15, 16 и 17.

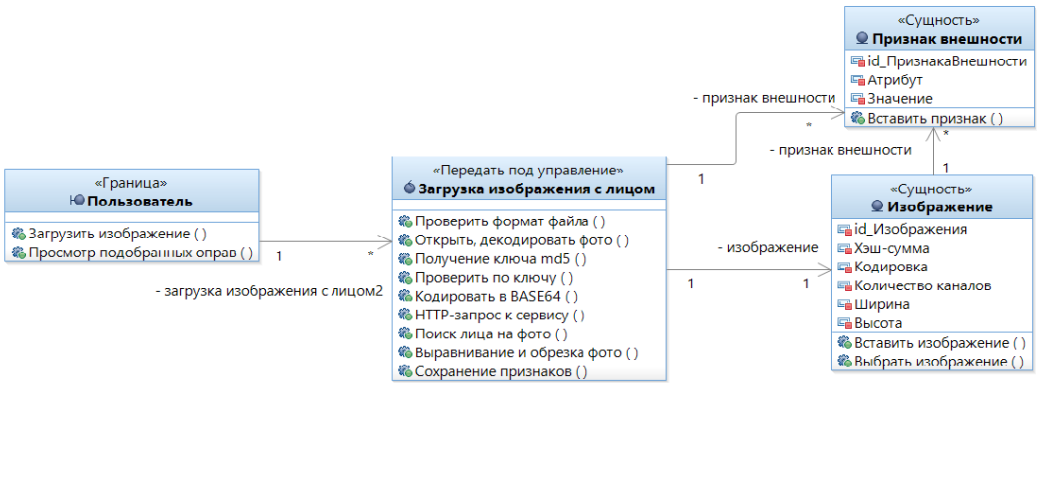


Рисунок 15. Диаграмма взаимодействия «Загрузка изображения с лицом»

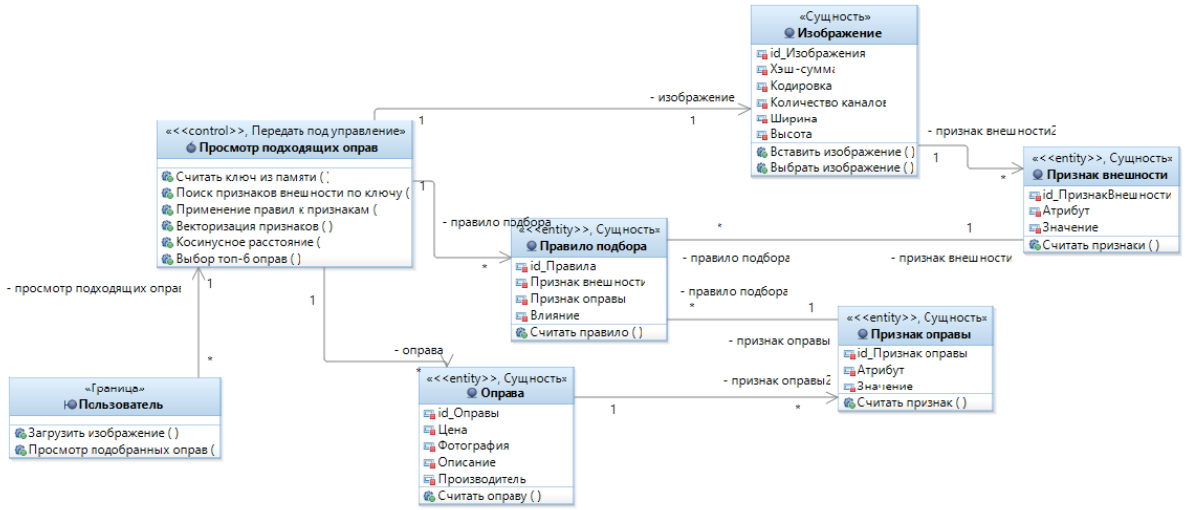


Рисунок 16. Диаграмма взаимодействия «Просмотр подходящих оправ»

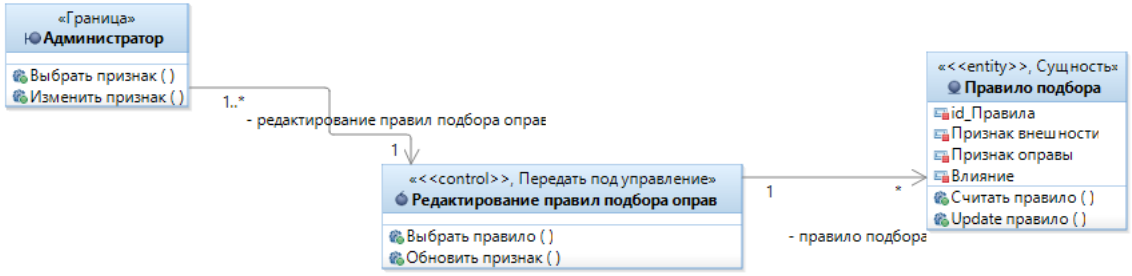


Рисунок 17. Диаграмма взаимодействия «Редактирование правил подбора оправ»

2.6. Пакеты анализа и сервисные пакеты в форме обобщенной диаграммы классов

Обобщенная диаграмма классов приведена на рисунке 18. Было выделено три пакета: Пакет пользователя, Пакет администратора и Пакет данных. Сущностные, граничные и управляющие классы были распределены между ними.

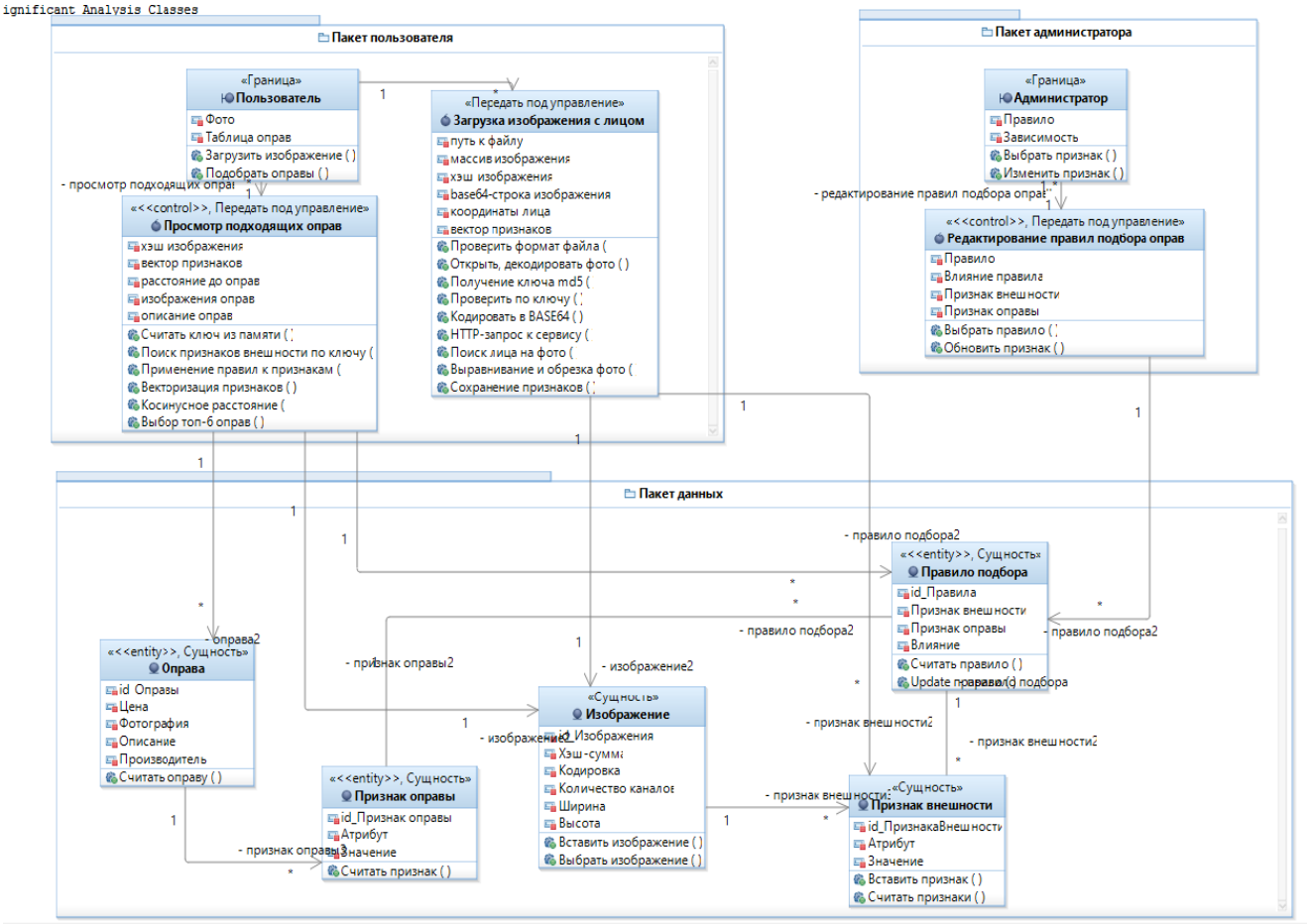


Рисунок 18. Диаграмма пакетов анализа

2.7. Трассировка пакетов в подсистемы

Трассировка пакетов в подсистемы изображена на рисунке 19. Пакет пользователя трассируется в две подсистемы, а именно клиентскую и серверную часть бизнес логики. Пакет администратора трассируется аналогично пакету пользователя. Серверные подсистемы реализуют паттерн работы с бизнес логикой модуль таблицы. Пакет данных трассируется в две подсистемы, где одна реализует паттерн работы с базой данных активная запись, а другая подсистема содержит саму базу данных.

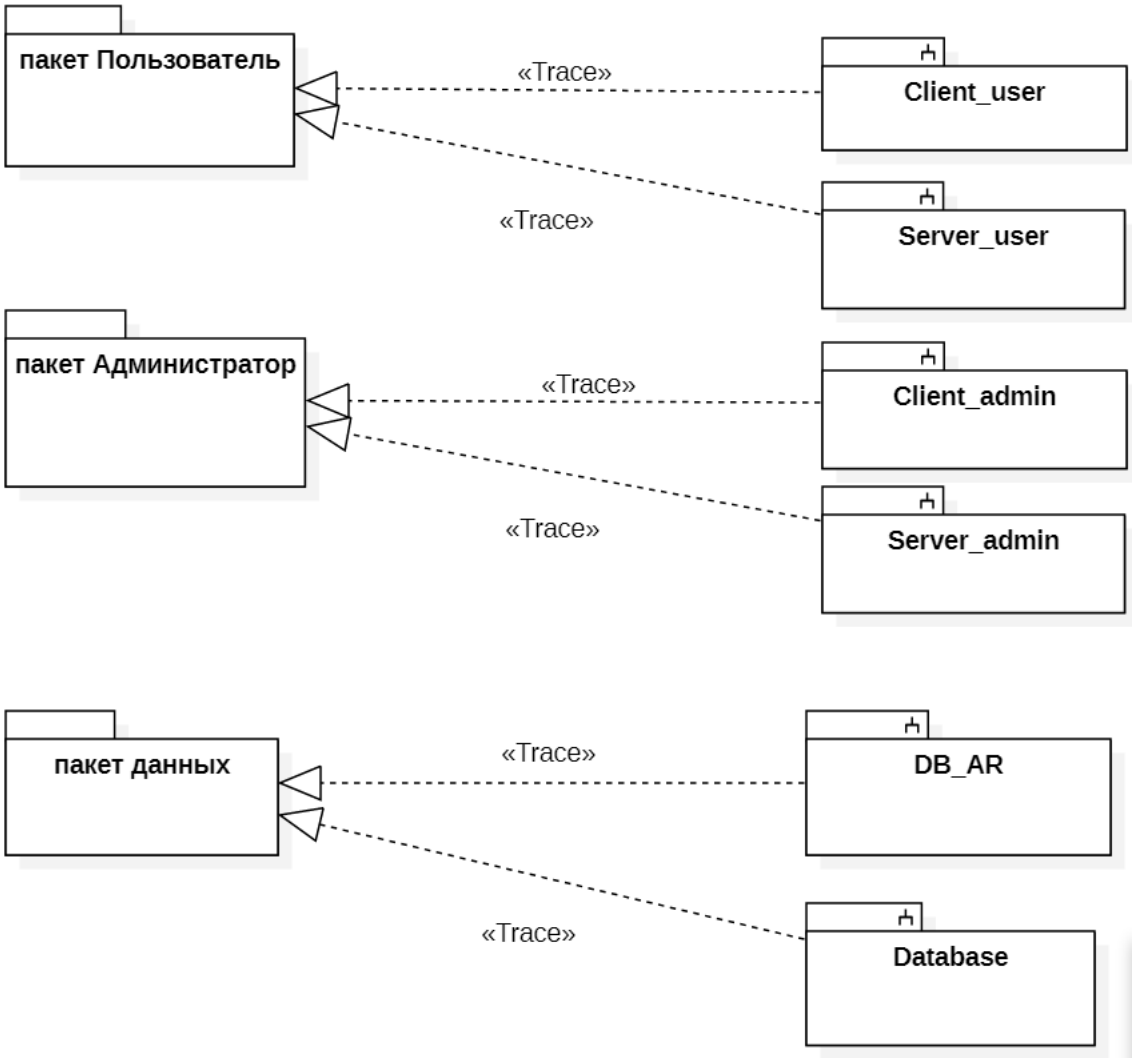


Рисунок 19. Трассировка пакетов в подсистемы

2.8. Трассировка классов анализа в классы проектирования

Трассировка классов анализа, управления и сущностей приведены на рисунках 20, 21 и 22 соответственно. Граничный класс Пользователь трассируется в три класса проектирования, которые будут динамически формироваться при помощи языка Python.

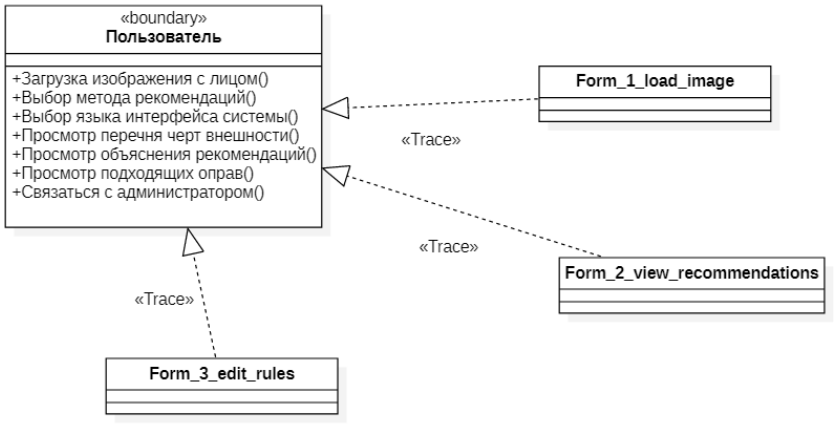


Рисунок 20. Трассировка классов анализа в классы проектирования

Трассировка управляющего класса приведена на рисунке 21 на примере класса «Загрузка изображения с лицом».



Рисунок 21. Трассировка управляющих классов в классы проектирования

Трассировка сущностных классов с учетом паттернов приведена на рисунке 22 на примере сущностного класса «Оправа».

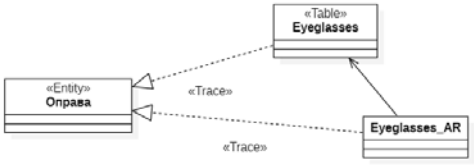


Рисунок 22. Трассировка сущностных классов в классы проектирования

2.9. Подсистемы, их уровни и зависимости

На рисунке 23 приведена уточненная диаграмма уровней подсистем с указанием зависимостей между ними. Как видно из диаграммы, для взаимодействия бизнес логики с базой данных используется подсистема DB_AR, реализующая паттерн работы активная запись.

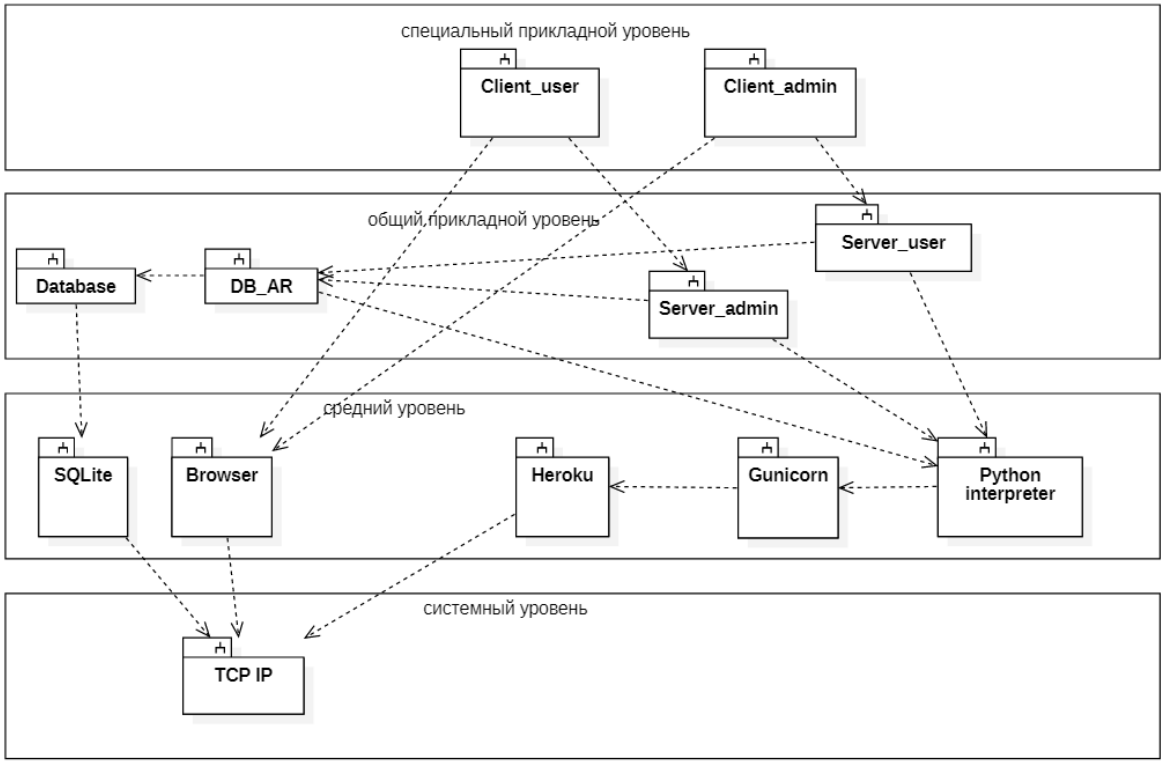


Рисунок 23. Диаграмма уровней подсистем

2.10. Диаграмма развертывания, узлы и линии связи, с указанием размещенных подсистем

На рисунке 24 представлена диаграмма развертывания системы с размещенными подсистемами. На стороне клиента находятся три артефакта: графический интерфейс клиента, администратора и браузер. На стороне сервера расположены шесть подсистем. На стороне сервера базы данных находятся две подсистемы.

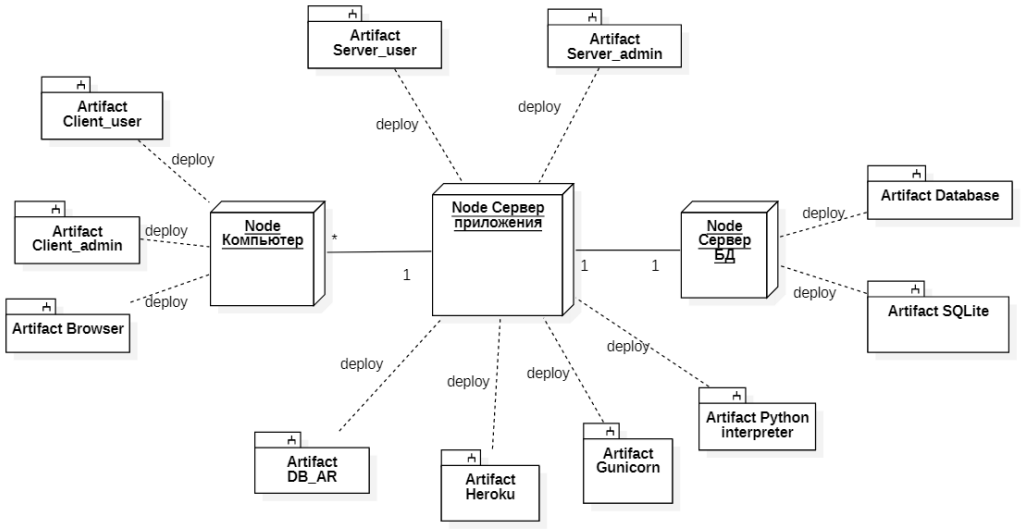


Рисунок 24. Диаграмма развертывания

2.11. Трассировка подсистем в компоненты

Была выполнена трассировка подсистем в компоненты, которую можно увидеть на рисунке 25.

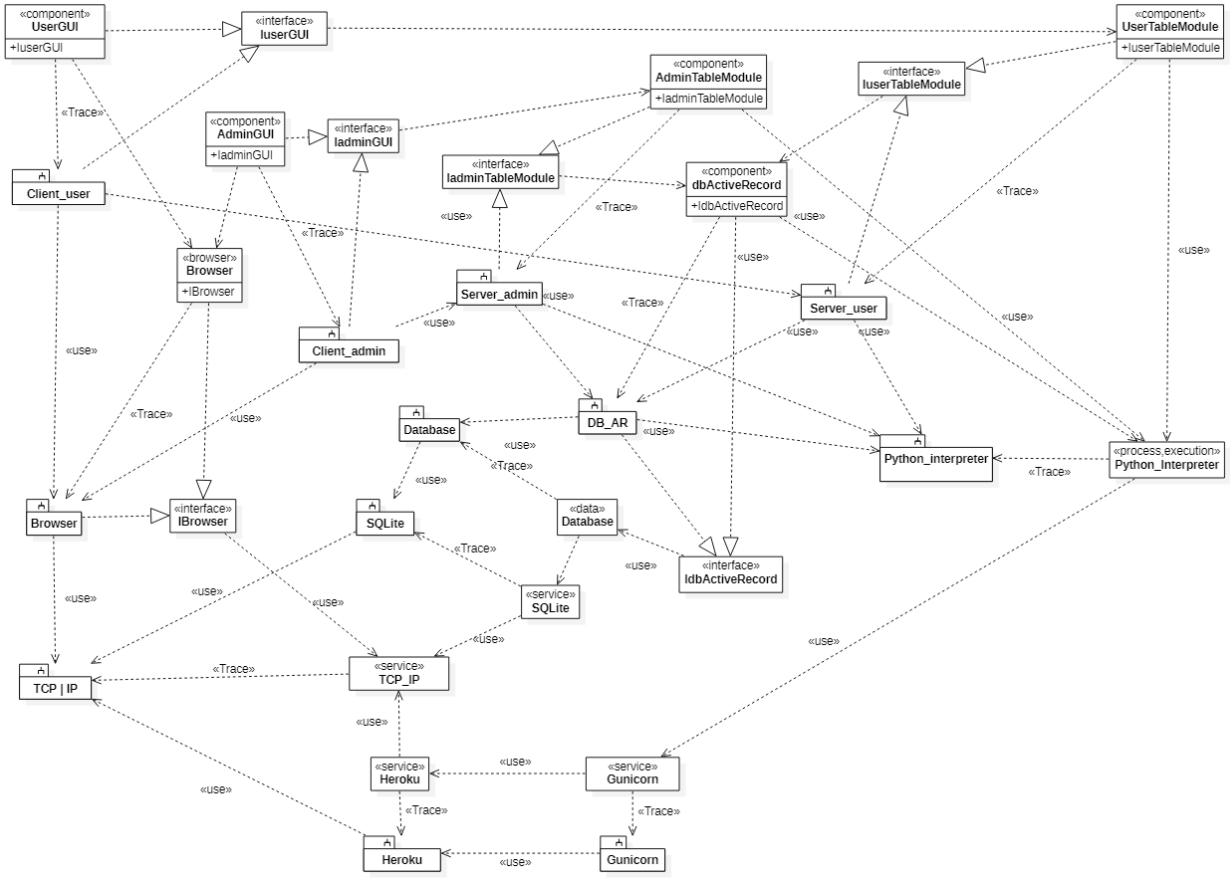


Рисунок 25. Трассировка подсистемы в компоненты

2.12. Трассировка классов проектирования в исходные файлы

На рисунке 26 представлена трассировка классов проектирования в исходные файлы. Исходные графические интерфейсы пользователя и администратора выделены в отдельные исходные файлы. Классы, реализующие бизнес-логику, трассируются для пользователя и администратора также в отдельные файлы. Все классы проектирования, реализующие паттерны БД, трассируются в один исходный файл. Таблицы базы данных трассируются в один исходный файл Database.db.

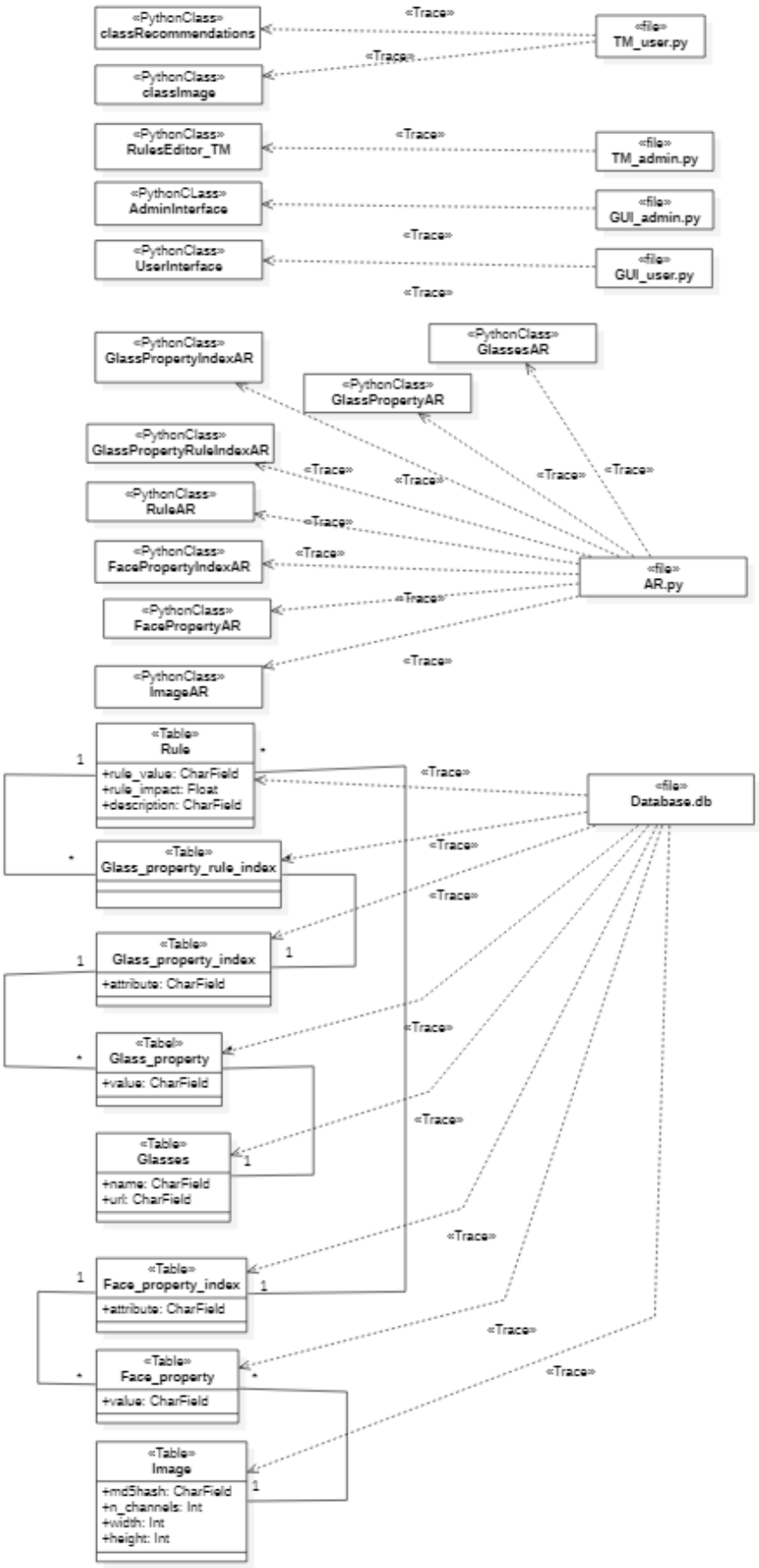


Рисунок 26. Трассировка классов проектирования в исходные файлы

2.13. Зависимость компонентов от исходных файлов

Были выявлены зависимости компонентов системы от исходных файлов. Данные зависимости можно увидеть на рисунке 27.

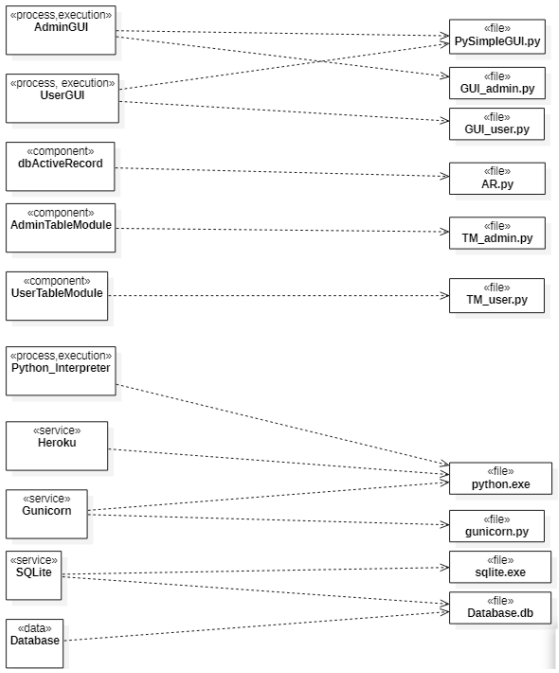


Рисунок 27. Зависимость компонентов от исходных файлов

2.14. Диаграммы компонентов и развертывания для реализации архитектуры

На рисунке 28 представлена диаграмма компонентов и развертывания для реализации архитектуры системы.

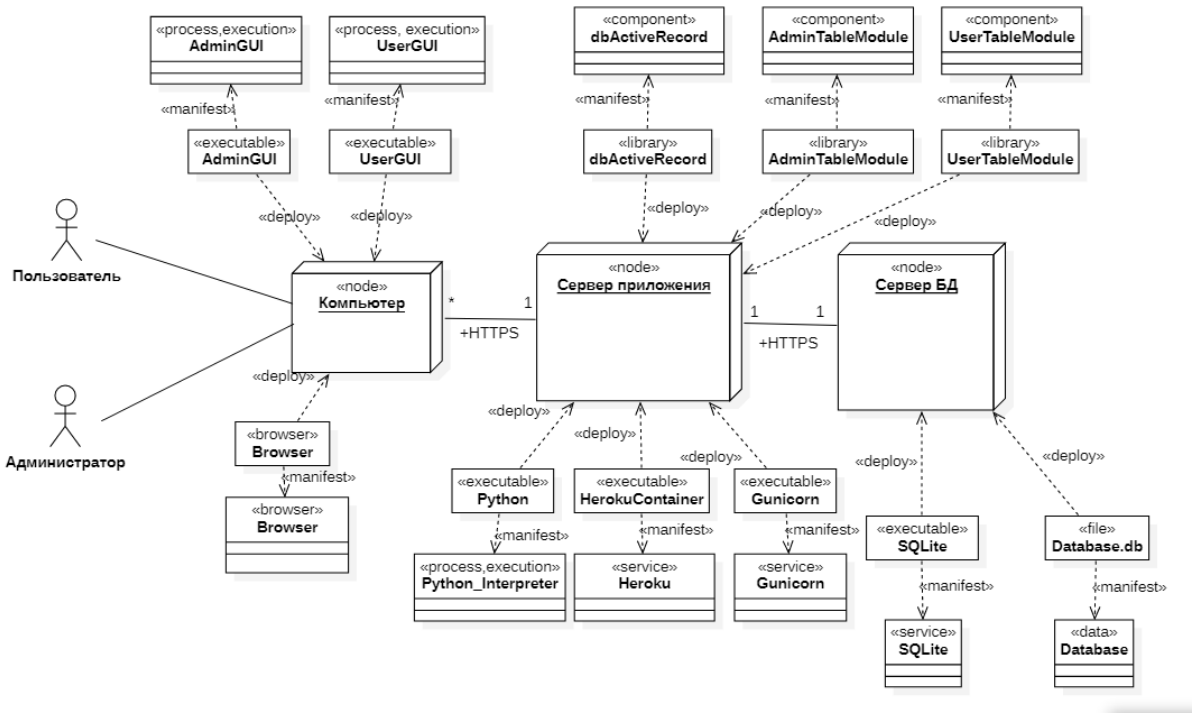


Рисунок 28. Диаграммы компонентов и развертывания для реализации архитектуры

2.15. Переработанный список рисков

Ранее сформированный список рисков (см. пункт 1.8 «Перечень критических рисков») был переработан. Результаты представлены в таблице 7.

Таблица 7. Переработанный список рисков

Риск	План управления	Удалось ли избежать
Критическая программная ошибка	Поиск и устранение ошибки	Большинство исключений покрыты тестами
Ошибка потери пользовательских данных	Поиск возможностей по созданию бекапов данных и дальнейшему восстановлению из них	Данные хранятся на энергонезависимых запоминающихся устройствах, требуется дальнейшее наблюдение за риском
Ошибки преобразования данных приводят к избыточным вычислением	Приведение типов с плавающей точкой к типам с фиксированной точкой для контроля погрешности вычислений.	Да
Низкая скорость отклика системы	Поиск возможностей по оптимизации работы системы	Да, используется кеширование запросов
Недостаточно понятный и малоэффективный интерфейс пользователя	Проведение А/Б тестирования для выявления простого и понятного интерфейса	Нет, не проводилось, требуется дальнейшее наблюдение за риском
Создание продукта, не требующегося на рынке	Прогнозирование необходимости продукта в будущем, в случае отсутствия каких-либо перспектив прекращение разработки	Пандемия COVID-19 не привела к понижению спроса на корректирующие оправы, требуется дальнейшее наблюдение за риском

2.16. Перечень итераций и их состав

Итерация 0:

Прецеденты:

- Загрузка изображения с лицом;
- Просмотр подходящих оправ.

Пакеты:

- Пакет данных;
- Пакет пользователя.

Подсистемы:

- Client_user;
- Server_user;
- DB_AR;
- Database.

Компоненты:

- UserGUI;
- UserTableModule;
- dbActiveRecord.

Итерация 1:

Прецеденты:

- Выбор языка интерфейса;
- Выбор метода рекомендаций;
- Просмотр правил подбора оправ;
- Редактирование правил подбора оправ;
- Просмотр объяснения рекомендаций.

Пакеты:

- Пакет данных (дополнено);
- Пакет администратора (дополнено);
- Пакет пользователя (дополнено).

Подсистемы:

- Client_user (дополнено);
- Server_user (дополнено);
- Client_admin;
- Server_admin.

Компоненты:

- UserGUI (дополнено);
- AdminGUI;
- UserTableModule (дополнено);
- AdminTableModule.

Итерация 2:

Прецеденты:

- Просмотр перечня черт внешности;
- Связаться с администратором.

Пакеты:

- Пакет администратора (дополнено);
- Пакет пользователя (дополнено).

Подсистемы:

- Client_user (дополнено);
- Server_user (дополнено);
- Client_admin (дополнено);

- Server_admin (дополнено).

Компоненты:

- UserGUI (дополнено);
- AdminGUI (дополнено);
- UserTableModule (дополнено);
- AdminTableModule (дополнено).

3. Этап построения

3.1. Диаграммы классов с учетом реализованных паттернов

С учётом реализованных паттернов бизнес-логики модуль таблицы и работы с базой данных активная запись была составлена диаграммы классов, которые представлена на рисунке 29.

Диаграммы последовательностей для основных прецедентов представлены на рисунках 30, 31.

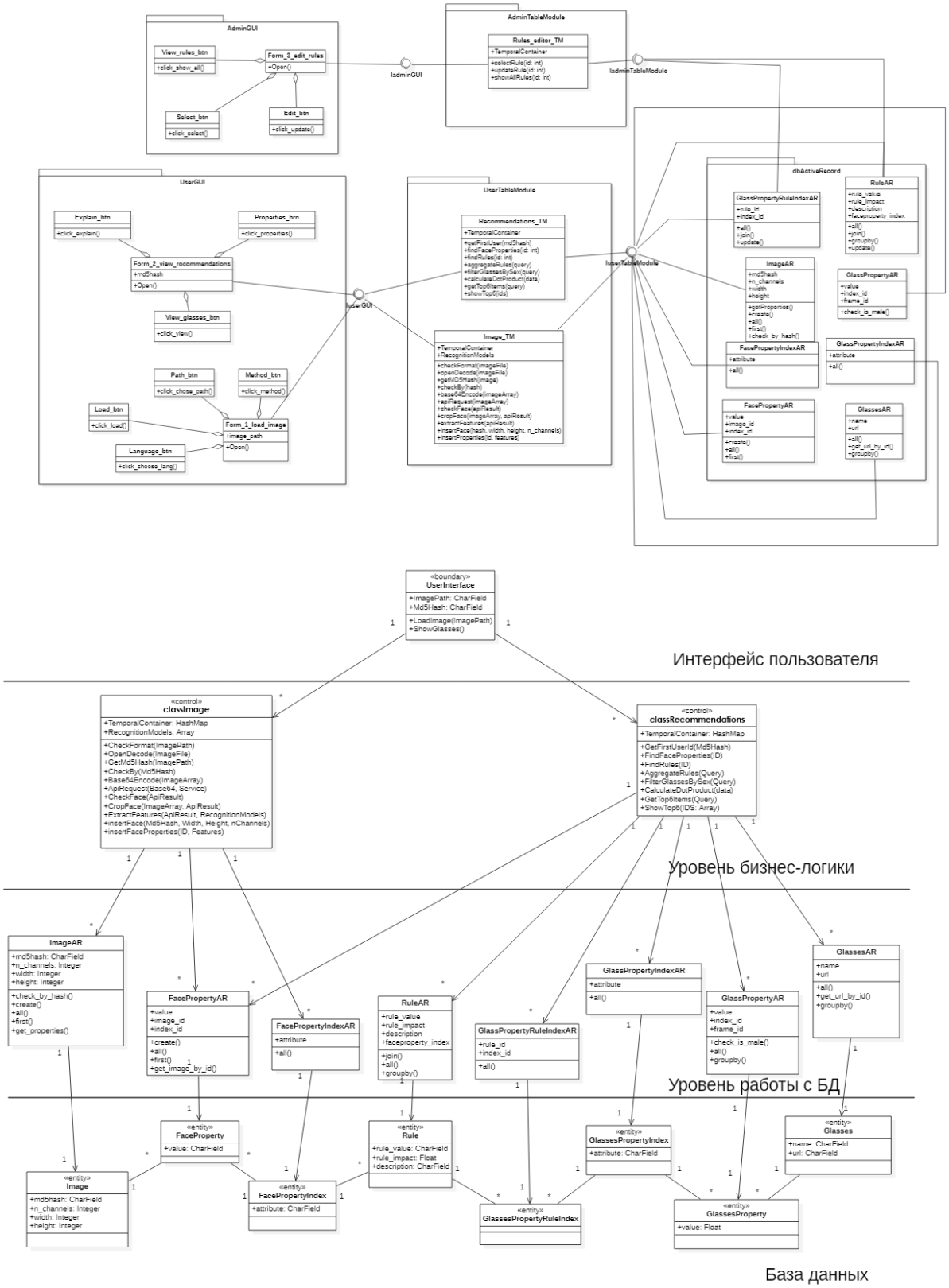


Рисунок 29. Диаграммы классов

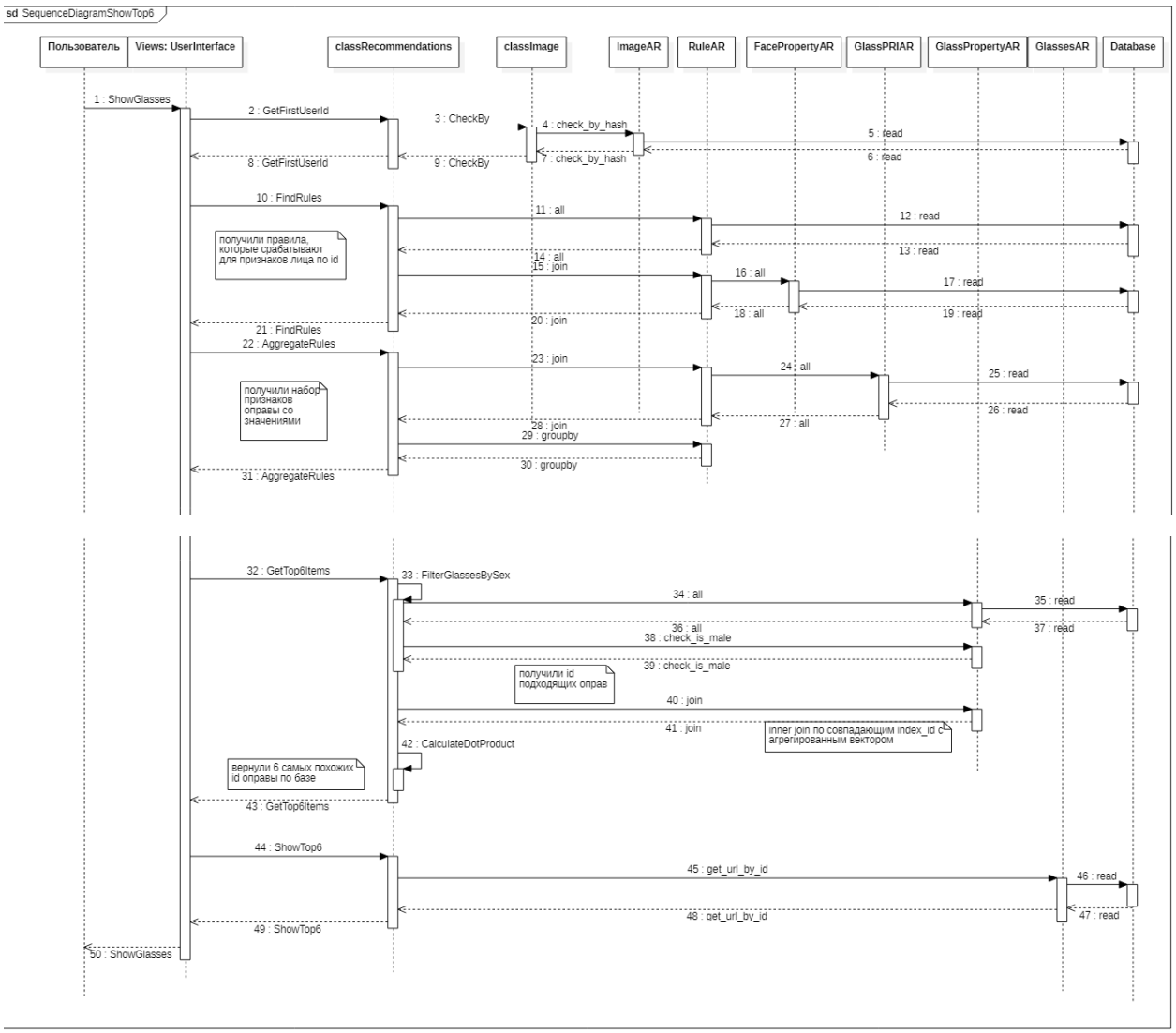


Рисунок 30. Диаграмма последовательностей «Просмотр подходящих оправ»

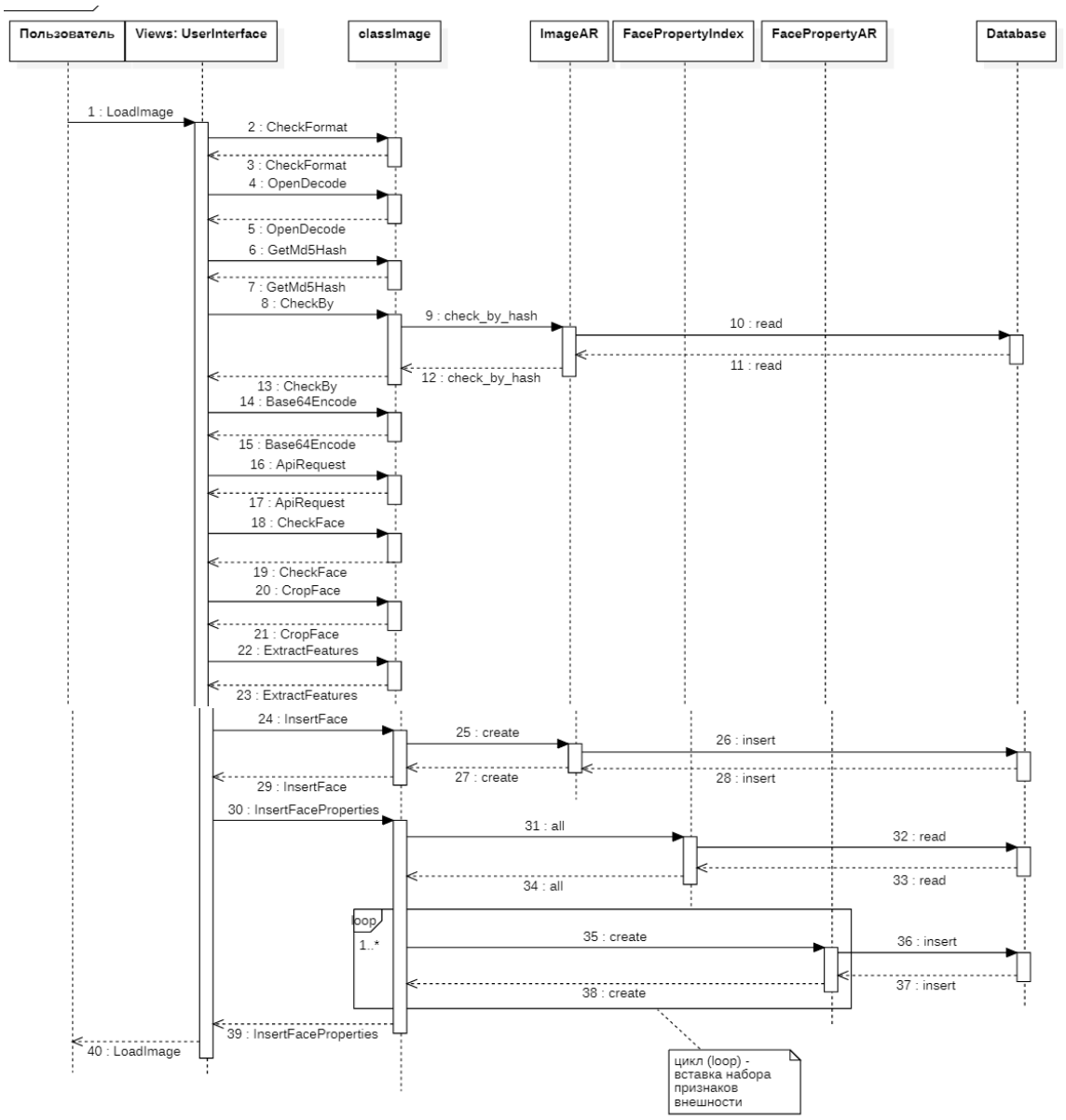


Рисунок 31. Диаграмма последовательностей «Загрузка изображения с лицом»

3.2. Диаграмма последовательностей в терминах подсистем

Была составлена диаграмма последовательностей в терминах подсистем для основного прецедента «Редактирование правил» (см. рисунок 32).

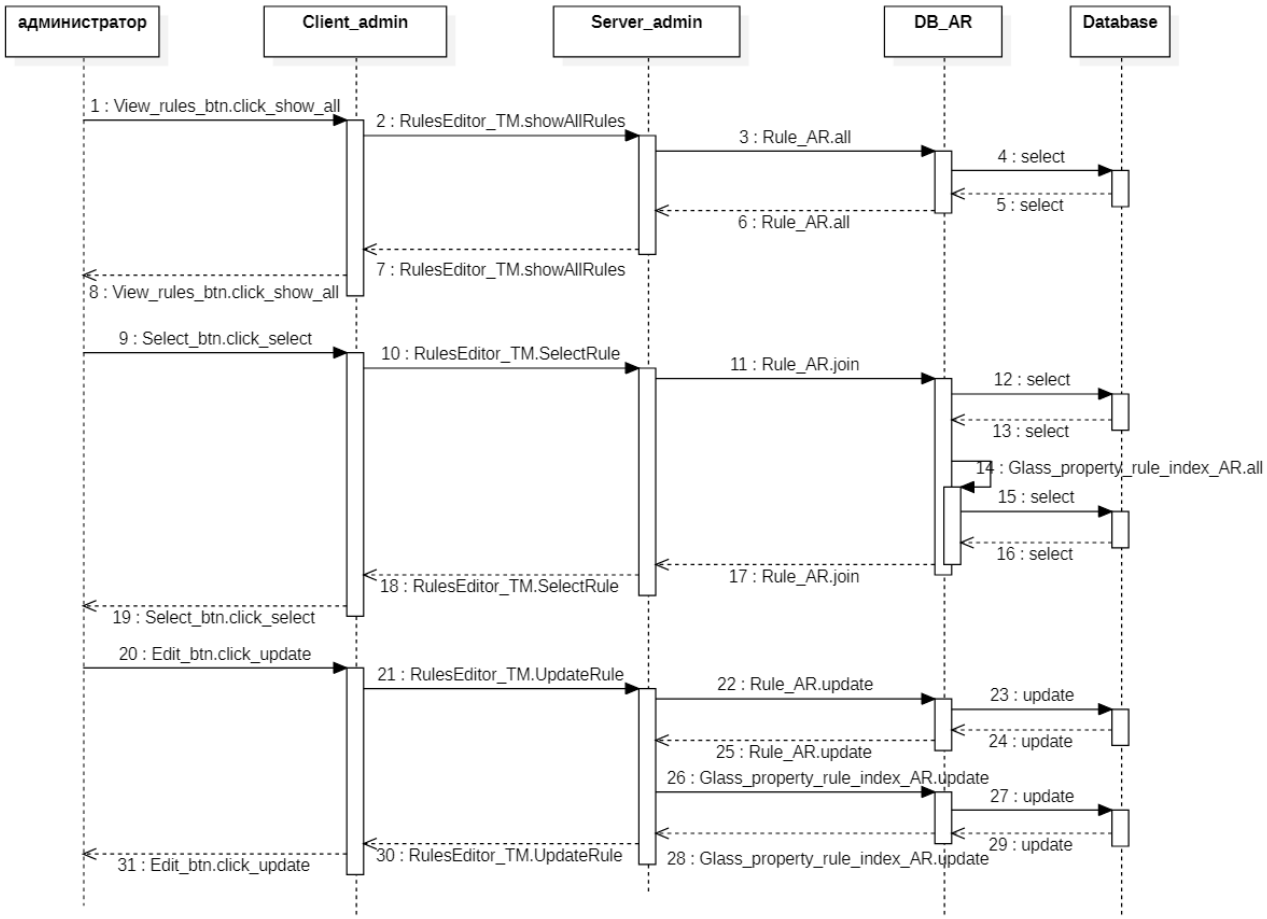


Рисунок 32. Диаграмма последовательностей прецедента «Редактирование правил» в терминах подсистем

3.3. Диаграмма последовательностей в терминах классов проектирования

Была составлена диаграмма последовательностей в терминах классов проектирования для основного прецедента «Редактирование правил» (см. рисунок 33).

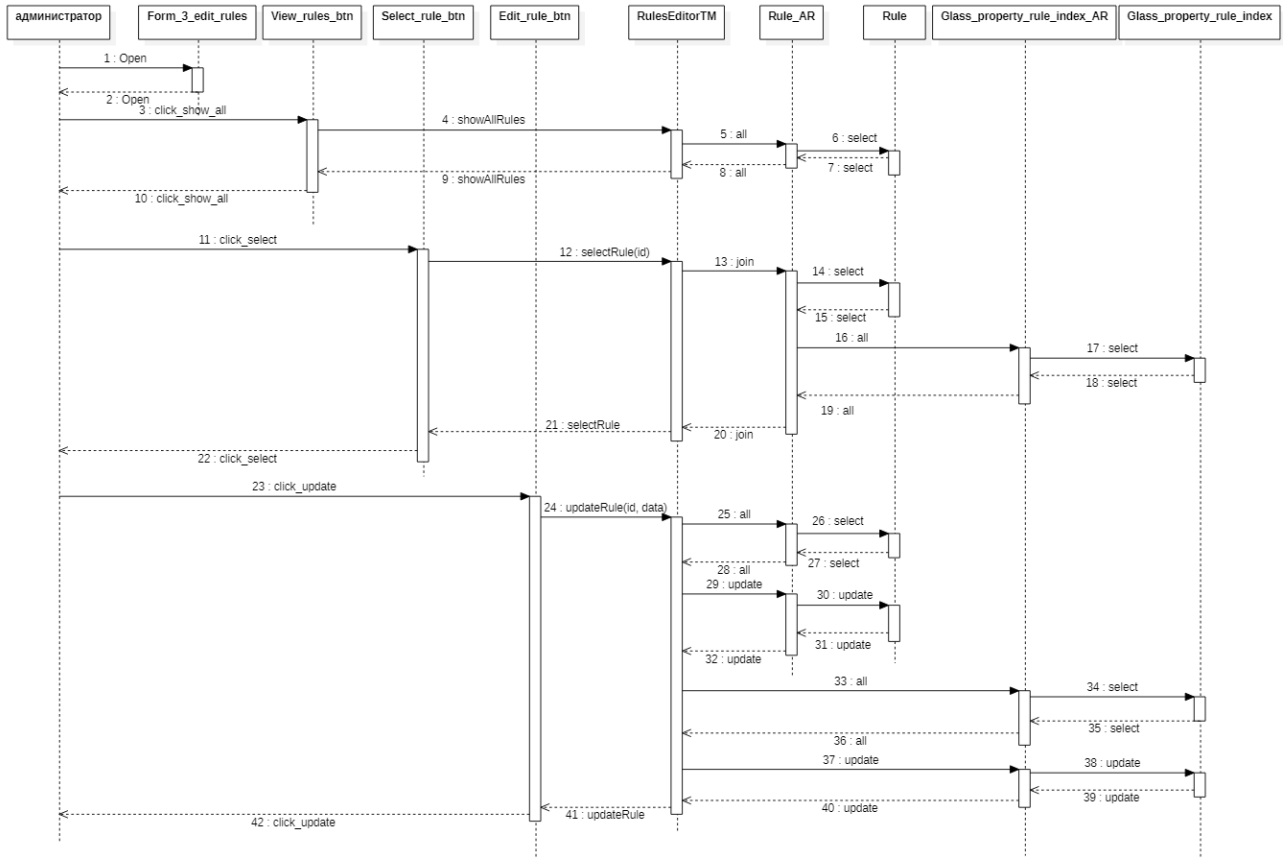


Рисунок 33. Диаграмма последовательностей прецедента «Редактирование правил» в терминах классов проектирования

3.4. Исходный код программы, реализующей архитектурно-значимые прецеденты

Был использован язык программирования Python, а также библиотека SQLAlchemy, которая изначально имеет паттерн Data mapper, паттерн активной записи был реализован «поверх» при помощи вспомогательной библиотеки sqlalchemy-mixins, заменяющую паттерн на активную запись по аналогии с популярным фреймворком Django. В результате были реализованы в декларативной форме восемь классов активной записи для взаимодействия с БД:

- GlassesAR;
- GlassPropertyAR;
- GlassPropertyIndexAR (GlassPIAR);
- GlassPropertyRuleIndexAR;
- RuleAR;
- FacePropertyIndexAR;
- FacePropertyAR;
- ImageAR.

Паттерн модуль таблицы был применен при программировании классов classImage и classRecommendations. Т.к. паттерн допускает обращение, объединения нескольких таблицам в рамках одного класса, то было решено ограничиться этими двумя.

Отличительной особенностью паттерна модуля таблиц является один экземпляр класса на все объекты таблицы. При этом идентификатор записи в таблице передается через методы, а сами методы зачастую статические [1].

Класс classImage реализует логику работы с изображением, алгоритмы выравнивания и распознавания, а также запросы к сервисам распознавания, внесение данных об изображении в базу данных. При этом он работает с базой данных через соответствующие методы классов активной записи.

Класс classRecommendations реализует логику работы с правилами рекомендаций, агрегации данных по признакам внешности, поиска оправ по их признакам с учётом пола. Аналогично классу classImage, для обращения к БД используются классы активной записи.

Класс classRecommendations

```
class classRecommendations:
    def __init__(self, logger=None, verbose=False):
        '''Constructor of recomendation class. In cobnstructor you should always define path
        to the photo with face

        Args:
            window (PySimpleGUI.Window class, optional): Copy of window instance,
            which is necessary for progress bar updates in GUI realization.
            logger (logging.Logger class, optional): Logger instance for logging events.
            verbose (bool, optional): If True, prints info messages.
        '''
        self._temporal_container = {'logger': logger,
                                    'verbose': verbose,
                                    'prefix': os.path.dirname(os.path.abspath(__file__)),
                                    'DBsession': session}


    @staticmethod
    def getFirstUserId(md5hash):
        return classImage.checkBy(md5hash)


    def findRules(self, face_id):
        s = self._temporal_container['DBsession'].query(RuleAR.id.label("rule_id")).\
                            join(FacePropertyAR,
                                 FacePropertyAR.index_id==RuleAR.f_properties_index_id).\
                            filter(FacePropertyAR.value==RuleAR.rule_value,
                                   FacePropertyAR.image_id==face_id)
        return s


    def aggregateRules(self, rule_ids):
        rule_ids = rule_ids.subquery('rule_ids')
        query = self._temporal_container['DBsession'].query(GlassPropertyRuleIndexAR.index_id,

func.sum(RuleAR.rule_impact).label("value")).\
                            join(rule_ids,
                                 RuleAR.id==rule_ids.c.rule_id).\
                            join(GlassPropertyRuleIndexAR,
                                 RuleAR.id==GlassPropertyRuleIndexAR.rule_id).\
                            group_by(GlassPropertyRuleIndexAR.index_id)
        return query


    def filterGlassesBySex(self, query, face_id):
        sex_value = ImageAR.find(face_id).is_male
        query = self._temporal_container['DBsession'].\
                        query(GlassesAR.id).\
                        join(GlassPropertyAR,
                             GlassesAR.id==GlassPropertyAR.frame_id)
        if sex_value:
            return query.filter(GlassesAR.is_male)
        return query.filter(GlassesAR.is_female)


    @staticmethod
    def __calculateDotProduct(data):
        return pd.DataFrame(data).\
                  groupby('gid').\
                  agg(lambda x: (x['gpvalue']*x['svalue']).sum()).\
                  reset_index()
```

```
    def getTop6Items(self, query, face_id):
        glasses_ids = self.filterGlassesBySex(query, face_id)

        s = query.subquery('s')
        gg = glasses_ids.subquery('gg')
        joined_data = self._temporal_container['DBsession'].\
                          query(gg.c.id.label('gid'),
                                GlassPropertyAR.value.label('gpvalue'),
                                s.c.value.label('svalue')).\
                          join(GlassPropertyAR,
                               gg.c.id==GlassPropertyAR.frame_id).\
                          join(s, GlassPropertyAR.index_id==s.c.index_id).\
                          filter(GlassPropertyAR.value > 0).\
                          all()

        # calculate dot product in RAM
        dotproduct = self.__calculateDotProduct(joined_data)

        # rank by dot product
        dotproduct.sort_values('svalue', ascending=False, inplace=True)

        # return top 6 ids
        return dotproduct.gid.tolist()[:6]


    def showTop6(self, glasses_ids):
        urls = self._temporal_container['DBsession'].query(GlassesAR.url).\
                                                     filter(GlassesAR.id.in_(glasses_ids)).\
                                                     all()

        pretty = lambda x: 'http:' + x if x[0] == '/' else x

        fig = plt.figure(figsize=(21, 14))
        axes = fig.subplots(2, 3, sharex='col', sharey='row')

        # two rows
        for i in range(2):
            # three columns
            for j in range(3):
                axes[i, j].text(200, 300, str(i * 3 + j + 1),
                                fontsize=18, ha='center')

                # download from internet
                img = io.imread(pretty(urls[i * 3 + j][0]))

                # guaranty that all images have the same ratio, if not, pad it with white color
                if img.shape[1] / img.shape[0] != 1.5:
                    offset = int((2/3 - img.shape[0] / img.shape[1]) * img.shape[1] // 2)
                    img = cv2.copyMakeBorder(img, offset, offset, 0, 0,
                                             cv2.BORDER_CONSTANT,
                                             value=(255, 255, 255))
                axes[i, j].imshow(cv2.resize(img, (375, 250)))
        plt.show(block=False)
```

Класс GlassesAR

```
class GlassesAR(BaseModel):
    __tablename__ = 'glasses'
    id = Column(Integer, primary_key=True)

    name = Column(String(130))
    url = Column(String(100))

    properties_ = relationship('GlassPropertyAR', back_populates='frame_')

    @hybrid_property
    def is_male(self):
        if self.properties_:
            return any([(prop.index_id == 31 and prop.value == 1) for prop in self.properties_])
        else:
            return False

    @is_male.expression
    def is_male(cls):
        return and_(GlassPropertyAR.value == 1, GlassPropertyAR.index_id == 31)
```

```
    @hybrid_property
    def is_female(self):
        if self.properties_:
            return any([(prop.index_id == 31 and prop.value == 0) for prop in self.properties_])
        else:
            return False

    @is_male.expression
    def is_female(cls):
        return and_(GlassPropertyAR.value == 0, GlassPropertyAR.index_id == 31)
```

Класс GlassPropertyAR

```
class GlassPropertyAR(BaseModel):
    __tablename__ = 'gproperties'

    id = Column(Integer, primary_key=True)
    value = Column(Numeric(2, 2))

    index_id = Column(Integer, ForeignKey('gproperties_indexes.id'))
    gindex_ = relationship('GlassPropertyIndexAR', back_populates='property_',
                           primaryjoin='GlassPropertyAR.index_id==GlassPropertyIndexAR.id')

    frame_id = Column(Integer, ForeignKey('glasses.id'))
    frame_ = relationship('GlassesAR', back_populates='properties_',
                          primaryjoin='GlassPropertyAR.frame_id==GlassesAR.id')
```

Класс GlassPropertyIndexAR

```
class GlassPropertyIndexAR(BaseModel):
    __tablename__ = 'gproperties_indexes'

    id = Column(Integer, primary_key=True)
    attribute = Column(String(100))

    property_ = relationship('GlassPropertyAR', back_populates='gindex_')
    rule_indexes_ = relationship('GlassPropertyRuleIndexAR', back_populates='rindex_')
```

Класс GlassPropertyRuleIndexAR

```
class GlassPropertyRuleIndexAR(BaseModel):
    __tablename__ = 'gproperties_rule_indexes'

    id = Column(Integer, primary_key=True)

    index_id = Column(Integer, ForeignKey('gproperties_indexes.id'))
    rindex_ = relationship('GlassPropertyIndexAR', back_populates='rule_indexes_',
                           primaryjoin='GlassPropertyRuleIndexAR.index_id==GlassPropertyIndexAR.id')

    rule_id = Column(Integer, ForeignKey('rules.id'))
    rule_ = relationship('RuleAR', back_populates='g_properties_index_',
                         primaryjoin='GlassPropertyRuleIndexAR.rule_id==RuleAR.id')
```

Класс RuleAR

```
class RuleAR(BaseModel):
    __tablename__ = 'rules'

    id = Column(Integer, primary_key=True)
    rule_value = Column(String(20))
    rule_impact = Column(Numeric(2, 2))

    g_properties_index_ = relationship('GlassPropertyRuleIndexAR', back_populates='rule_')

    f_properties_index_id = Column(Integer, ForeignKey('fproperties_indexes.id'))
    f_properties_index_ = relationship('FacePropertiesIndexAR', back_populates='frule_',

primaryjoin='RuleAR.f_properties_index_id==FacePropertiesIndexAR.id')
    description = Column(String(100))
```

Класс FacePropertiesIndexAR

```
class FacePropertiesIndexAR(BaseModel):
    __tablename__ = 'fproperties_indexes'

    id = Column(Integer, primary_key=True)
    attribute = Column(String(20))

    frule_ = relationship('RuleAR', back_populates='f_properties_index_')

    face_property_ = relationship('FacePropertyAR', back_populates='face_property_index_')
```

Класс FacePropertyAR

```
class FacePropertyAR(BaseModel):
    __tablename__ = 'fproperties'

    id = Column(Integer, primary_key=True)
    value = Column(String(20))

    index_id = Column(Integer, ForeignKey('fproperties_indexes.id'))
    face_property_index_ = relationship('FacePropertiesIndexAR',
                                        back_populates='face_property_',

primaryjoin='FacePropertyAR.index_id==FacePropertiesIndexAR.id')

    image_id = Column(Integer, ForeignKey('images.id'))
    image_ = relationship('ImageAR',
                          back_populates='face_properties_',
                          primaryjoin='FacePropertyAR.image_id==ImageAR.id')
```

Класс ImageAR

```
class ImageAR(BaseModel):
    __tablename__ = 'images'

    id = Column(Integer, primary_key=True)
    md5hash = Column(String(64))

    n_channels = Column(Integer())

    width = Column(Integer())
    height = Column(Integer())
    face_properties_ = relationship('FacePropertyAR', back_populates='image_')

    @hybrid_property
    def is_male(self):
        if self.face_properties_:
            return any([(prop.index_id == 17 and prop.value == 'male') for prop in
self.face_properties_])
        else:
            return False

    @is_male.expression
    def is_male(cls):
        return and_(FacePropertyAR.value == 'male', FacePropertyAR.index_id == 17)

    @hybrid_method
    def check_by_hash(self, md5hash):
        if self.md5hash == md5hash:
            return self.id
        else:
            return False

    @check_by_hash.expression
    def check_by_hash(cls, md5hash):
        return case([
            (cls.md5hash == md5hash, cls.id),
        ], else_=False)
```

3.5. Описание тестовых вариантов для тестирования модулей

Основной прецедент рекомендательной системы: «Загрузка изображения с лицом». При тестировании будем рассматривать работу пользователя напрямую с методами системы.

Загрузка в систему изображения по его файловому пути на локальном устройстве или ссылке в Интернете, возвращает сообщение и md5-ключ изображения.

Предусловие:

- Пользователь запустил систему;
- Есть доступ в Интернет.

Пользователь (через форму) обращается к методу загрузки изображения с лицом в систему.

Входные параметры метода:

- image_path – файловый путь изображения на локальном устройстве или

ссылка на изображение в Интернете.

Выходные параметры метода:

- message – строка с сообщением о результате;
- md5hash – строка, содержащая ключ изображения в системе.

При этом существует ряд правил для успешного выполнения загрузки:

- есть доступ в Интернет;
- image_path – строка формата «*.*»;
- расширение (символы после точки в image_path) совпадает с одним из поддерживаемых расширений;

- изображение уже могло быть ранее загружено в систему;
- минимальная сторона декодированного изображения не менее 128 пикселей, а максимальная не более 3000 пикселей;
- на изображении в действительности находится не менее одного лица (в случае нескольких лиц метод использует первое лицо слева сверху для анализа).

Таким образом, Постусловия:

- Найдено по ключу => message = «ОК», md5hash = ключ фото в системе;
- Неподдерживаемый формат файла => message = «Ошибка, описание», md5hash = пустая строка;
- Неподдерживаемые размеры изображения => message = «Ошибка, описание», md5hash = пустая строка;
- Нет лица на фотографии => message = «Ошибка, описание», md5hash = пустая строка;
- Успешная операция => message = «ОК», md5hash = ключ фото в системе.

Было составлено разбиение на классы эквивалентности (КЭ), которое представлено в таблице 8.

Таблица 8. Разбиение на КЭ

Параметр	Варианты значений правил	
Система запущена	Да	Нет
Есть Интернет	Да	Нет
image_path формата «*.*»	Да	Нет
Расширение image_path	В списке поддерживаемых	Нет
Хэш изображения найден	Нет	Да
Минимальная сторона изображения	128…+∞ пикселей	1…127 пикселей
Максимальная сторона изображения	1…3000 пикселей	3001... +∞ пикселей
Количество лиц на изображении	1 и более	0

Количество тестов для проверки граничных значений будет равен количеству границ. Рекомендуется проверять значения вплотную к границе. Получили 3 границы КЭ для параметров «Минимальная сторона изображения», «Максимальная сторона изображения», «Количество лиц на изображении».

Если условия ввода задают дискретное множество [3], то тесты для:

- Min и max из значений;
- Чуть < min и чуть > max.

Значит нужно провести следующие тесты для этих параметров, оставив остальные параметры валидными:

1. Изображение размером 127xN, 128xN ,129xN пикселей;
2. Изображение размером 2999xN, 3000xN, 3001xN пикселей;
3. Изображение без лица, с одним лицом, с двумя лицами.

Т.е. проверяем крайние допустимые и недопустимые значения. Пример успешного тестового варианта представлен в таблице 9. Пример провального же тестового варианта приведен в таблице 10.

Таблица 9. Успешный тестовый вариант

Параметр	Вариант	Значение
Система запущена	Да	-
Есть Интернет	Да	-
image_path формата «*.*»	Да	Image.png
Расширение image_path	В списке поддерживаемых	PNG
Хэш изображения	Не найден	2a1c8614279d35528ab9a4d18e56b784
Минимальная сторона изображения	128…+∞ пикселей	129
Максимальная сторона изображения	1…3000 пикселей	800
Количество лиц на изображении	1 и более	1

Входные данные:

image_path= 'Image.png'

***Выходные данные*:**

message= 'Successully loaded image to the database!'

md5hash='2a1c8614279d35528ab9a4d18e56b784'

Таблица 10. Провальный тестовый вариант

Параметр	Вариант	Значение
Система запущена	Да	-
Есть Интернет	Да	-
image_path формата «*.*»	Да	Image.png
Расширение image_path	В списке поддерживаемых	PNG
Хэш изображения	Не найден	2a1c8614279d35528ab9a4d18e56b784
Минимальная сторона изображения	128…+∞ пикселей	127
Максимальная сторона изображения	1…3000 пикселей	800
Количество лиц на изображении	1 и более	1

Входные данные:

image_path= 'Image.png'

***Выходные данные*:**

message='Unsupported image size!'

md5hash=''

3.6. Перечень и последовательность проведения тестов

Последовательность

Проведение тестов будем выполнять в следующем порядке:

- Модульное тестирование;
- Нагрузочное тестирование;
- Тестирование интерфейса.

Перечень

Будем тестировать класс UserInterface, а конкретно метод loadImage (см. рисунок 38). Тесты будут реализованы с помощью библиотеки pytest, используемой для создания модульных тестов. Реализуем функциональный тест, проверяющий входные и выходные данные метода loadImage. Тестовые варианты изображены на рисунке 34. Пример модульных тестов для метода loadImage показан на рисунке 35. Результат успешного модульного тестирования приведен на рисунке 36.

```
from tempfile import TemporaryDirectory
import pytest
import gc
import os
from shutil import copy
from trpohw2_GUI import UserInterface


# Initialize inputs and expected outputs for test suits
# For load image method tests
loader_init_cases = [('A', ['Unsupported image format!', '']),
                     ('examples/127x1024.jpg', ['Unsupported image size!', '']),
                     ('examples/3001x1024.jpg', ['Unsupported image size!', '']),
                     ('examples/3000x1024.jpg',
                      ['Successully loaded image to the database!', 'c95a3cc00dee766e9e8db1dcc6c46b0b']),
                     ('examples/128x1024.jpg',
                      ['Successully loaded image to the database!', '4224f5c8330c216ec652f93d3c72b0d1']),
                     ('examples/cat.jpg', ['No face found on image', '']),
                     ('examples/exists.jpeg',
                      ['Image already exists in the database', '089e2bb329bda31fb6e66f49ed7972b6']),
                     ('examples/00bbc7df21452970cb689c0979325c31.jpeg',
                      ['Successully loaded image to the database!', '2a1c8614279d35528ab9a4d18e56b784'])]
```

Рисунок 34. Задание тестовых случаев для модульного тестирования

```python
@pytest.yield_fixture()
def ins_setup():
    ''' Initialize resource fixture as temporary directory
    instance for each test

    '''
    print('user interface setup')
    with TemporaryDirectory() as tmp_folder:
        copy('data/database.db', os.path.join(tmp_folder, 'database.db'))
        ins = UserInterface()
        yield ins
        ins.close()
        copy(os.path.join(tmp_folder, 'database.db'), 'data/database.db')
    # teardown
    # call garbage collector
    gc.collect()


@pytest.mark.parametrize(('image_path', 'expected_answer'), loader_init_cases)
def test_init(ins_setup, image_path, expected_answer):
    ''' Test suits for loadImage method
    '''
    print('Testing path: %s' % image_path)
    print('Testing answer: %s' % expected_answer)
    answer = ins_setup.loadImage(image_path)
    print(answer)
    assert len(answer) == 2
    assert answer[0] == expected_answer[0]
    assert answer[1] == expected_answer[1]
```

Рисунок 35. Пример модульных тестов

```
C:\Games\glasses_expert\expertglasses>pytest -vv --cov=./ -cov-report=html trpohw2_tests.py
================================= test session starts =================================
platform win32 -- Python 3.7.0, pytest-6.1.2, py-1.9.0, pluggy-0.13.1 -- c:\users\денис\appdata\local\progra
cachedir: .pytest_cache
rootdir: C:\Games\glasses_expert\expertglasses, configfile: ov-report=html
plugins: asyncio-0.14.0, cov-2.10.1
collected 8 items

trpohw2_tests.py::test_init[A-expected_answer0] PASSED                           [ 12%]
trpohw2_tests.py::test_init[examples/127x1024.jpg-expected_answer1] PASSED       [ 25%]
trpohw2_tests.py::test_init[examples/3001x1024.jpg-expected_answer2] PASSED      [ 37%]
trpohw2_tests.py::test_init[examples/3000x1024.jpg-expected_answer3] PASSED      [ 50%]
trpohw2_tests.py::test_init[examples/128x1024.jpg-expected_answer4] PASSED       [ 62%]
trpohw2_tests.py::test_init[examples/cat.jpg-expected_answer5] PASSED            [ 75%]
trpohw2_tests.py::test_init[examples/exists.jpeg-expected_answer6] PASSED        [ 87%]
trpohw2_tests.py::test_init[examples/00bbc7df21452970cb689c0979325c31.jpeg-expected_answer7] PASSED [100%]
```

Рисунок 36. Результат модульного тестирования

Покрытие тестами

Рассмотрим покрытие проекта тестами с помощью библиотеки pytest-coverage. Получим с ее помощью статистику о покрытии кода (см. рисунок 37).

```
---------- coverage: platform win32, python 3.7.0-final-0 -----------
Name                                    Stmts   Miss  Cover
-------------------------------------------------------------
faceplusplus.py                            92     49    47%
model_architecture.py                      11      0   100%
shufflenet_and_gans\shufflenetv2.py        65     24    63%
shufflenet_and_gans\utils.py               59      1    98%
trpohw2_AR.py                              96     13    86%
trpohw2_GUI.py                            106     41    61%
trpohw2_TM.py                             338     58    83%
trpohw2_tests.py                           28      0   100%
-------------------------------------------------------------
TOTAL                                     795    186    77%

====================== 8 passed, 14 warnings in 1107.99s (0:18:27) ====
```

Рисунок 37. Отчёт о покрытии кода тестами

```
def loadImage(self, image_path):
    if not self.__image_table_module.checkFormat(image_path):
        return 'Unsupported image format!', ''

    image = self.__image_table_module.openDecode(image_path)

    md5hash, width, height, n_channels = self.__image_table_module.getMd5Hash(image)

    if min(width, height) < 128 or max(width, height) > 3000:
        return 'Unsupported image size!', ''

    if self.__image_table_module.checkBy(md5hash):
        return 'Image already exists in the database', md5hash

    b64string = self.__image_table_module.base64Encode(image)
    apiresult = self.__image_table_module.apiRequest(b64string, 'first')

    if not self.__image_table_module.checkFace(apiresult):
        return 'No face found on image', ''

    res = self.__image_table_module.cropFace(image, apiresult)
    b64string = self.__image_table_module.base64Encode(res['image'])
    tags = self.__image_table_module.apiRequest(b64string, 'second')

    face_vector = self.__image_table_module.extractFeatures(res['image'], res['eyes'],
                                                            res['features'], tags)

    face_id = self.__image_table_module.insertFace(md5hash, width, height, n_channels)
    self.__image_table_module.inserFaceProperties(face_id, face_vector)
    return 'Successully loaded image to the database!', md5hash
```

Рисунок 38. Тестируемая функция системы

Отчет производительности системы

Производительность вебсайта будем рассматривать с помощью ChromeDevTools в браузере. Для реальных условий выберем эмулируемую Сеть – 3G Fast, а частоту процессора, порезанную в 4 раза. В ходе тестирования, нажимались всевозможные кнопки, вбивались значения в поля и так далее. Результат тестирования представлен на рисунке 39.

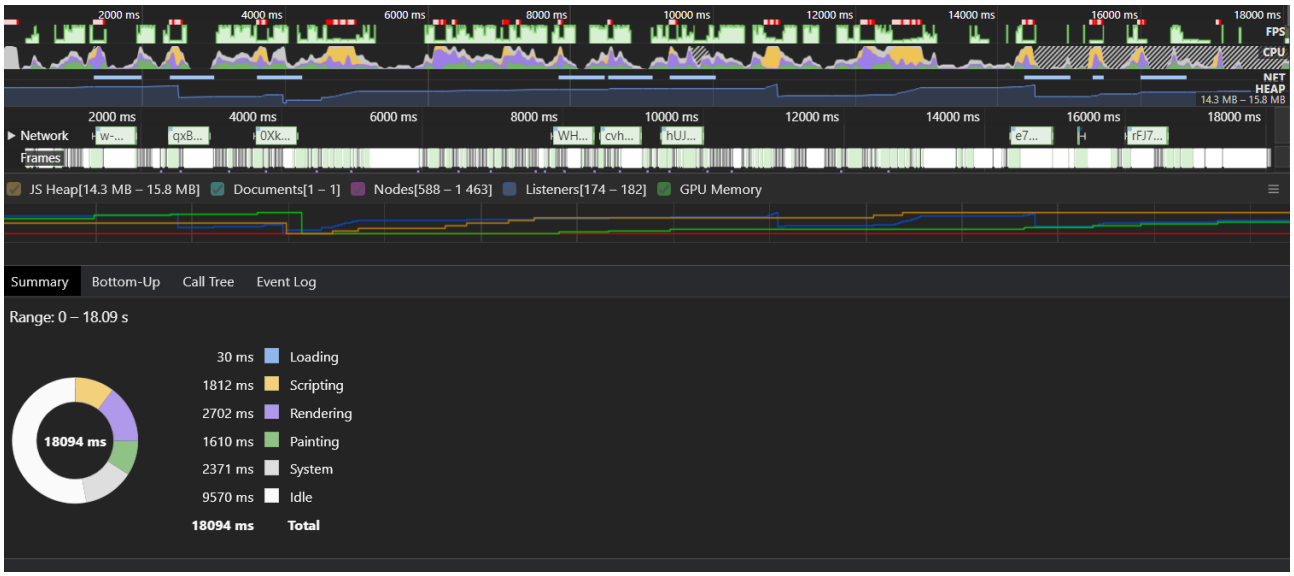


Рисунок 39. Отчёт теста производительности ChromeDevTools

Рассмотрим полученные результаты:

1. Загрузка (Loading) страницы заняла 30 мс;
2. Выполнение и загрузка скриптов (Scripting) страницы занимает 1812 мс;
3. Рендеринг, а именно отрисовка страницы (Rendering) страницы занял 2702 мс;
4. Погрузка css - стилей страницы (Painting) страницы занял 1610 мс;
5. Погрузка системных браузерных настроек (System) страницы занял 2371 мс;
6. Время простоя системы после выполнения всех процессов до окончания тестирования приложения (Idle) страницы занял 9570 мс;

Исходя из результатов, мы можем судить о довольно быстром приложении.

Использование памяти клиентского приложения приведено на рисунке 40. Как можно увидеть, используется 14 МБ оперативной памяти, что говорит о низких требованиях к устройствам, на которых будет использоваться система.

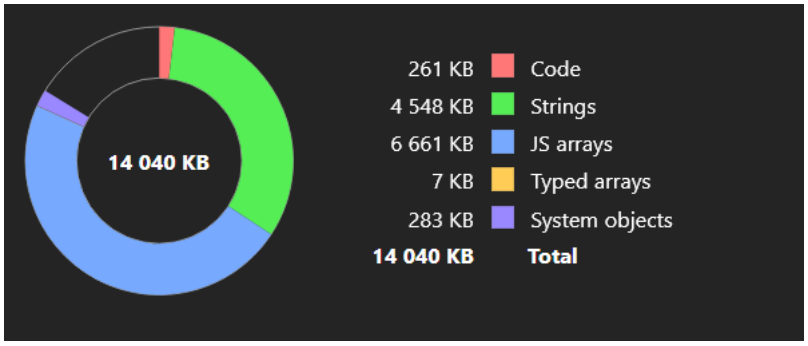


Рисунок 40. Использование памяти приложения

Тестирование графического интерфейса

Интеграционное тестирование GUI будем выполнять с помощью Selenium на Python. Построим сценарий ввода пользователем пути к изображению и нажатия кнопки «Отправить» (см. рисунок 41). Будем считать тест выполненным успешно, если на обновленной странице будет обнаружена фраза «ТОП-6 Оправ согласно». Результат интеграционного тестирования продемонстрирован на рисунке 42.

```
from selenium import webdriver
import time as t
import pytest
import os


@pytest.yield_fixture()
def driver_setup():
    chrome_options = webdriver.ChromeOptions()
    prefs = {}# {"profile.managed_default_content_settings.images": 2}
    chrome_options.add_experimental_option("prefs", prefs)
    driver = webdriver.Chrome(executable_path='chromedriver.exe',
                              chrome_options=chrome_options)
    yield driver
    driver.close()


def test_init(driver_setup):
    driver = driver_setup
    driver.get('http://glasses-expert.herokuapp.com/')
    t.sleep(7)
    driver.find_element_by_id("imagefile").send_keys(os.path.abspath('examples/exists.jpeg'))
    t.sleep(7)
    driver.find_element_by_id("submit").click()
    t.sleep(7)
    driver.find_element_by_id("meta2").click()
    driver.find_element_by_id("meta2").click()
    t.sleep(3)
    assert driver.find_element_by_id("mcol2").text.strip().startswith('ТОП-6 Оправ согласно')
```

Рисунок 41. Тестирование графического интерфейса

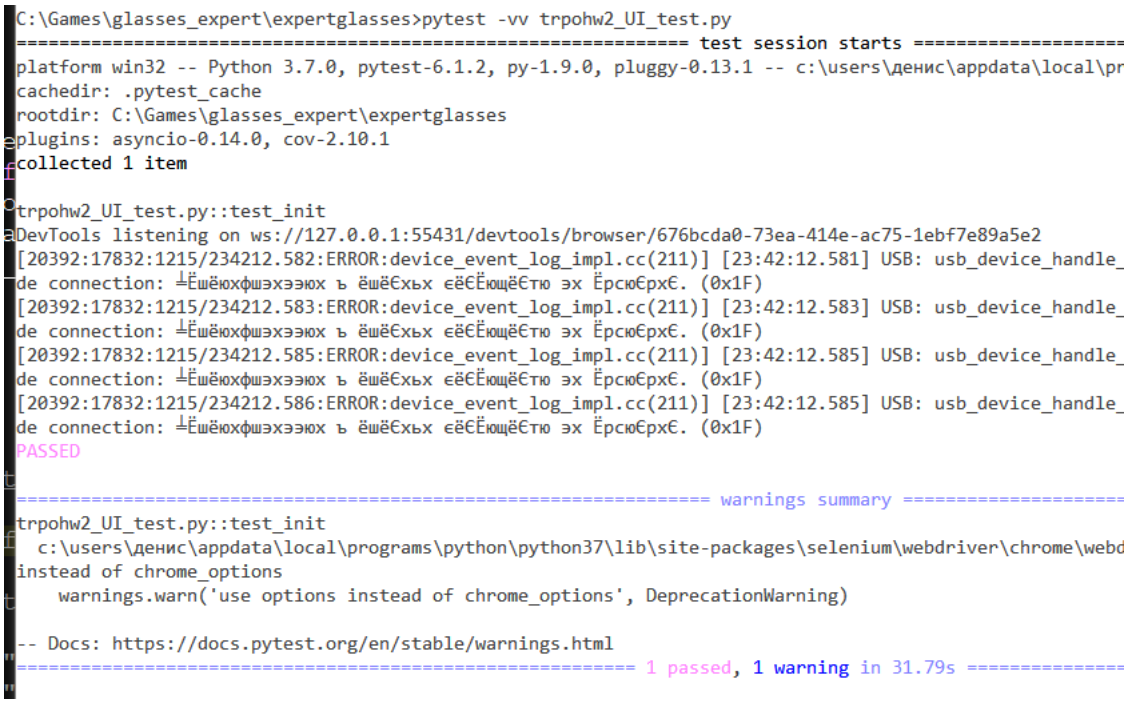


Рисунок 42. Результат успешного тестирования

4. Этап внедрения

4.1. Перечень программ и рекомендации по установке

Перечень программ разработанной системы:

1) Приложение – это исполнимый файл формата .exe для Windows и .deb для Linux.

Рекомендации по установке:

- Интернет: подключение к интернету;
- ОС: Windows 7+, Ubuntu LTS2014+;
- ОЗУ: >256 МБ;
- HDD: 512-1024 МБ на жестком диске.

Первоначальная загрузка модулей (нейронные сети) занимает от 30 секунд до минуты в зависимости от оборудования. Загрузка изображения (Load Image) с последующим извлечением признаков занимает ~5 секунд. Поиск по базе (Show Glasses) занимает ~10 секунд.

4.2. Перечень документации для пользователей и заказчиков

Для пользователей будет предоставлено руководство по пользованию приложением. Заказчику передается: составленное по его требованиям техническое задание, программа, руководство по эксплуатации [4].

4.3. Рекомендации по внедрению

Для внедрения продукта в магазины оптики рекомендуется провести курсы повышения классификации среди сотрудников с последующей сертификацией.

Список использованных источников

Список использованных источников:

1. Унифицированный процесс разработки программного обеспечения: учебное пособие / Виноградова М.В., Белоусова В.И. – М.: МГТУ им.Н.Э. Баумана. – 2015 г. http://ebooks.bmstu.ru/catalog/193/book1303.html
2. Фаулер М. Архитектура корпоративных приложений. - М.:Изд.дом Вильямс. - 2008 г.
3. Орлов С.А. Технологии разработки программного обеспечения. - СПб: Питер, - 2002 г.
4. Якобсон А, Дуч Г., Рамбо Дж. Унифицированный процесс разработки программного обеспечения. - Спб.: Питер. - 2002 г.